

Redis

高级面试题完整答案

从数据结构、缓存一致性与持久化到高可用、集群和性能排障

高级 Java / 分布式系统面试复习讲义

2026-06

阅读说明

本讲义面向具有 Java 后端、微服务、中间件或分布式系统经验的中高级工程师。内容不是把 Redis 简化成“内存版 HashMap”，而是从一次命令如何执行、对象如何编码、数据如何过期和淘汰开始，逐步推导到缓存一致性、分布式锁、持久化、复制、Sentinel、Cluster、线上排障和系统设计。

每道题按真实面试中更容易获得高评价的顺序组织：**结论先行** -> **底层机制** -> **示例或代码** -> **生产边界** -> **常见误区** -> **面试追问**。回答时不应只背命令，应说明为什么这样设计、故障发生时失去什么保证，以及如何监控和验证。

版本语境

- 本文以 **Redis Open Source 8.x** 的通用机制为主，并覆盖大量生产环境仍在使用的 Redis 6.x/7.x。Redis 8 将原 Redis Stack 的搜索、JSON、概率数据结构等能力整合进统一发行版；这些能力是否可用仍应以实际部署版本和发行方式为准。
- 截至 2026 年 6 月，Redis 官方文档已经发布 Redis 8.8 相关说明。本文不会把 8.8 新增命令当作所有环境都具备的默认能力；版本敏感点会单独标注。
- 文中的“主节点/副本”对应历史文档中的 master/replica；“Sentinel”指 Redis Sentinel；“Cluster”指 Redis Cluster，而不是第三方代理分片方案。
- Java 示例以 Lettuce、Jedis 和 Redisson 的通用使用原则为主。客户端 API 可能随版本变化，面试重点是连接、超时、重试、拓扑刷新和幂等语义。

推荐的面试回答结构

1. **一句话结论**：先回答能否做到、保证到什么程度。
2. **数据路径**：说明 key、对象、事件循环、日志、复制或路由如何流转。
3. **失败窗口**：指出宕机、网络分区、超时、重试和切换时的边界。
4. **工程方案**：给出限流、降级、监控、容量和恢复策略。
5. **验证方法**：用指标、命令、压测或故障演练证明结论。

目录

1. Redis 是什么，为什么快，哪些场景不适合使用
2. 常用数据类型如何选型，复杂度和业务场景是什么
3. Redis 对象与内部编码如何实现，为什么会自动转换
4. Redis 真的是单线程吗，事件循环与 I/O 线程如何工作
5. 大 Key、热 Key 和高危命令有什么区别，如何治理
6. Key 的过期删除和内存淘汰是如何工作的
7. Pipeline、事务、Lua 与 Redis Functions 应如何选择
8. Redis 事务为什么不支持传统回滚，WATCH 能解决什么

9. 如何设计缓存读写模式并处理数据库一致性
10. 缓存穿透、击穿、雪崩和缓存污染如何治理
11. 如何实现一个正确的 Redis 分布式锁
12. Redis 限流器应如何设计
13. 排行榜、计数器、延时任务和滑动窗口如何实现
14. Bitmap、HyperLogLog、Bloom Filter、Top-K 如何选型
15. Pub/Sub、Streams 与专业消息队列有什么差异
16. Redis Streams 消费组、ACK、PEL、重试和幂等如何设计
17. RDB 与 AOF 的机制、取舍和恢复顺序是什么
18. fork、写时复制和持久化为何可能造成延迟与内存峰值
19. Redis 主从复制、全量同步和部分同步如何工作
20. Sentinel 如何完成故障判断、选主与切换
21. Redis Cluster 的槽、重定向、Hash Tag 和扩缩容如何工作
22. Redis Cluster 能保证强一致吗，网络分区会发生什么
23. Redis 内存为什么会碎片化，如何做内存优化与容量规划
24. Redis 延迟突然升高或 CPU/内存异常时如何系统排查
25. Java 客户端连接、序列化、超时、重试和故障切换如何设计
26. 场景设计：构建高并发商品缓存、库存与异步事件系统
27. 附录：高频追问速答、排障命令、上线清单与官方资料

第一部分：基础原理与数据结构

1. Redis 是什么，为什么快，哪些场景不适合使用？

1.1 结论先行

Redis 是以内存为主要数据介质、提供丰富数据结构和原子命令的键值数据平台。它常被用于缓存、计数器、排行榜、会话、限流、锁、实时状态和轻量消息场景，但它不是关系型数据库的简单替代品。

Redis 快的核心不是单一原因，而是多个因素叠加：

- 数据主要在内存中，避免了传统磁盘随机访问。
- 命令执行路径短，数据结构针对常用操作优化。
- 大多数核心命令在主事件循环中串行执行，减少线程切换和锁竞争。
- 使用事件驱动网络模型，可同时管理大量连接。
- RESP 协议简单，支持长连接、Pipeline 和批量命令降低 RTT。
- Redis Cluster 可通过分片把吞吐扩展到多个主节点。

“内存数据库”并不等于“任何命令都很快”。单线程执行意味着一个 $O(N)$ 命令、大对象删除、Lua 长脚本或 fork 抖动都可能拖慢同一实例上的其他请求。

1.2 一次命令的简化执行链路

```
Java Client
-> TCP connection / RESP request
-> event loop reads request
-> parse command and arguments
-> execute command against keyspace
-> update expire / persistence / replication state
-> build response
-> socket write
```

真正的端到端延迟包括客户端排队、连接池等待、网络 RTT、Redis 输入缓冲、命令执行、输出缓冲和客户端反序列化。SLOWLOG 只记录服务端命令执行时间，不包含网络传输和客户端等待，因此“客户端 500ms、SLOWLOG 没记录”并不矛盾。

1.3 典型适用场景

- **Cache Aside 缓存**：商品详情、配置、权限、聚合结果。
- **原子计数**：点赞、库存预扣、访问量、序列号。
- **排序集合**：排行榜、优先级队列、时间窗口。
- **集合关系**：共同关注、标签、去重、在线用户。
- **会话和临时状态**：验证码、幂等 Token、登录态。
- **实时事件流**：Streams、轻量任务通知。

- 概率统计：HyperLogLog、Bloom Filter、Top-K。

1.4 不适合或需要谨慎的场景

需要复杂关系与强约束

Redis 不提供关系型外键、任意 Join、声明式约束和完整 SQL 事务。订单、账户余额等核心事实通常仍应以数据库或可靠账本为最终权威。

单对象非常大

一个几十 MB 的 String、包含数百万成员的 Hash/ZSet，会造成网络阻塞、复制压力、迁移困难、删除延迟和内存峰值。Redis 更擅长大量大小受控的对象，而不是把大文件塞进一个 key。

要求零数据丢失的强一致主存储

开源 Redis 复制通常是异步的，主节点确认写入后仍可能在故障切换窗口丢失尚未复制的数据。AOF 每次 fsync 可缩小本地宕机数据丢失窗口，但不能自动把异步复制变成共识协议。

需要无限历史消息和复杂消息治理

Streams 能满足不少实时事件场景，但对于跨地域复制、长期海量日志、复杂消费者治理、严格顺序分区和成熟生态，Kafka、Pulsar、RabbitMQ 等专业消息系统通常更合适。

1.5 面试中的高质量表达

Redis 快是“内存 + 合适的数据结构 + 短执行路径 + 事件驱动 + 批量降低 RTT”的综合结果。单线程只减少了锁竞争，并不保证每个命令都快。选型时我会先明确 Redis 是缓存、协调组件还是事实存储，再讨论持久性、数据丢失窗口、对象大小和故障降级。

1.6 常见误区

- **误区：Redis 只有缓存用途。** 它还提供流、搜索、JSON、概率结构等能力，但不代表应把所有中间件职责都集中到 Redis。
- **误区：Redis 单线程所以不会有并发问题。** 单条命令原子不等于跨命令业务原子；网络超时、客户端重试和主从切换仍会产生竞态。
- **误区：QPS 高就一定要 Redis。** 如果数据无法失效、不能接受旧值，或者每次都要复杂查询，增加缓存可能只会引入一致性成本。

2. Redis 常用数据类型如何选型，复杂度和业务场景是什么？

2.1 结论先行

选择 Redis 数据类型的原则是：让业务操作直接对应服务端原子命令，避免在客户端“取出整个对象 -> 修改 -> 写回”。同一个需求可由多种类型实现，应同时比较时间复杂度、内存、并发原子性、范围查询和集群路由。

2.2 核心类型对比

类型	典型操作	常见复杂度	典型场景	主要风险
String	GET/SET/INCR/MGET	$O(1)$ ；MGET 与 key 数有关	缓存、计数、位图、分布式锁值	大 Value、全量序列化
Hash	HGET/HSET/HINCRBY	平均 $O(1)$	对象字段、聚合计数	单 Hash 过大、无法天然给旧版本逐字段 TTL
List	LPUSH/RPOP/LRANGE	端点 $O(1)$ ，按索引访问可能 $O(N)$	栈、队列、最近记录	中间访问、超长列表
Set	SADD/SISMEMBER/SINTER	单元素平均 $O(1)$ ，集合运算与规模相关	去重、关系、共同集合	大集合交并集阻塞
Sorted Set	ZADD/ZRANGE/ZRANK	常见 $O(\log N)$ 或 $O(\log N+M)$	排行榜、延时任务、滑动窗口	member/score 设计、热点集中
Bitmap	SETBIT/BITCOUNT	位操作高效	签到、状态位、布尔人群	稀疏大 offset 浪费空间
HyperLogLog	PFADD/PFCOUNT	近似统计	UV、基数估算	有误差，不能枚举元素
Stream	XADD/XREADGROUP/XACK	与读取条数、组状态有关	事件流、消费者组	PEL 堆积、保留策略
JSON/Query	文档路径读写、查询	依命令与索引而定	结构化文档和实时查询	版本、索引内存、复杂查询成本

复杂度必须结合返回数据量理解。例如 ZRANGE 0 -1 即使内部定位高效，返回一百万个成员仍会消耗 CPU、网络 and 客户端内存。

2.3 String 与 Hash 如何选择

假设用户资料包含 20 个字段：

整对象 String：

```
user:1001 -> JSON / protobuf bytes
```

优点：一次 GET，序列化格式灵活，适合整体读写。缺点：修改一个字段也需要读改写；多个客户端并发更新不同字段可能覆盖；大对象网络成本高。

Hash：

```
HSET user:1001 name Alice level 7 city Hangzhou
HINCRBY user:1001 score 10
```

优点：字段级原子更新、只取需要字段。缺点：字段很多时管理复杂；是否支持字段级过期取决于 Redis 版本和客户端能力，跨版本设计仍应保守。

选择建议：整体读写、需要压缩或统一版本控制时偏向 String；多个字段独立更新、计数较多时偏向 Hash。不要为了“节约 key 数”把所有用户塞进一个超级 Hash。

2.4 List、Stream 和 Sorted Set 如何选择队列

- **List**：简单 FIFO，可使用阻塞弹出；消息被弹出后需要应用自行处理失败恢复。
- **Stream**：自带消息 ID、消费者组、待确认列表和 ACK，适合需要消费跟踪的轻量事件流。
- **Sorted Set**：按 score 排序，适合延时任务或优先级队列，但抢占、重试和确认机制要自己设计。

若业务要求严格消息可靠性、跨机房长期堆积和复杂消费治理，优先评估专业 MQ，而不是仅因为已有 Redis 就复用。

2.5 Set 与 Sorted Set 的边界

集合只关心唯一性和成员关系时用 Set，例如角色用户集合。需要排序、范围、排名或时间窗口时用 Sorted Set。例如：

```
ZADD rank:2026 9520 user:1001
ZREVRANK rank:2026 user:1001
ZRANGE rank:2026 0 99 REV WITHSCORES
```

Score 是 IEEE 754 双精度浮点数。若把超大整数时间戳和序号直接拼到 score 中，需要确认精度边界；也可把时间作为 score、唯一序号放在 member 中，并设计同分排序规则。

2.6 面试追问

问：为什么不能用 KEYS 扫描业务数据后在客户端聚合？

KEYS 会遍历整个键空间并集中返回，可能阻塞服务和制造大响应。应使用明确索引结构、SCAN 渐进遍历或离线数据平台；SCAN 也不是强一致快照，遍历期间数据变化可能导致重复或遗漏。

问：Hash 能否显著节约内存？

小 Hash 可能使用紧凑 listpack 编码，确实比大量独立 key 节约元数据。但超过阈值后会转成哈希表，且超级 Hash 会带来热 Key、迁移和过期粒度问题。内存优化不能只看 key 数。

3. Redis 对象与内部编码如何实现，为什么会自动转换？

3.1 结论先行

Redis 对外暴露 String、Hash、List、Set、ZSet、Stream 等抽象类型，内部会根据元素数量、元素长度和值特征选择不同编码。小对象通常采用紧凑连续结构节省内存；规模增长后转换为更通用、更新更稳定的数据结构。

因此，回答“ZSet 底层是跳表”并不完整：小 ZSet 可使用 listpack，达到阈值后才采用 skiplist 等通用编码。真实编码可通过 OBJECT ENCODING key 观察。

3.2 Redis Object 的作用

Redis key 指向一个对象，对象元数据通常包含：

- 逻辑类型，例如 String 或 ZSet。
- 内部编码，例如 int、embstr、raw、listpack、hashtable、skiplist。
- LRU/LFU 相关信息。
- 引用计数等管理信息。

这种“类型与编码分离”的设计使外部 API 保持不变，同时允许内部根据数据规模优化。

3.3 各类型常见编码

逻辑类型	常见内部编码	说明
String	int / embstr / raw	整数可直接编码；短字符串紧凑分配；长或可修改字符串使用 raw
List	quicklist / listpack	quicklist 由多个紧凑节点组成，平衡内存和局部更新
Hash	listpack / hashtable	小 Hash 紧凑存储，扩大后转换为字典
Set	intset / listpack / hashtable	纯小整数集合可用 intset；新版本小集合可用 listpack
Sorted Set	listpack / skiplist	小集合紧凑编码，大集合需要跳表支持范围与排名
Stream	radix tree + listpack	使用基数树组织 ID 区间，叶子内紧凑存储条目

版本差异非常重要：Redis 6.2 及更早版本常见 ziplist，Redis 7.0 后许多场景改为 listpack；Redis 7.2 后 Set 也可出现 listpack。面试时应说“依版本和阈值变化”，不要死背旧实现。

3.4 SDS 为什么比 C 字符串更适合 Redis

Redis 字符串底层使用 SDS（Simple Dynamic String）思想，而不是直接把所有内容当作以 \0 结尾的 C 字符串。其价值包括：

- 单独记录长度，获取长度不必扫描。
- 可存储二进制数据，内容中可包含零字节。

- 管理容量，减少频繁扩容。
- API 更容易做边界检查，降低缓冲区问题。

但 SDS 仍然占用元数据和分配器空间。大量很短的 key/value 可能出现“业务数据只有几字节，实际内存几十字节”的情况。

3.5 跳表为何适合 Sorted Set

ZSet 同时要支持：

- 按 member 找 score。
- 按 score 排序和范围扫描。
- 按排名访问。
- 插入和删除。

通用编码通常结合字典和跳表：字典便于按 member 查找，跳表便于有序操作。跳表实现简单，范围扫描自然，平均插入/查询 $O(\log N)$ 。

```
level 3: head -----> 80
level 2: head -----> 30 -----> 80
level 1: head -> 10 -> 30 -> 50 -> 80
```

小 ZSet 使用 listpack 是因为元素少时连续存储更节省内存，线性移动的成本可接受；规模扩大后转换可以避免后续操作退化。

3.6 编码转换的工程影响

- 转换会消耗 CPU，并可能在数据跨越阈值时出现延迟抖动。
- 不要依赖内部编码作为业务契约；升级后阈值或实现可能变化。
- 容量测试必须使用与生产相似的 key 长度、value 大小和元素分布。
- MEMORY USAGE key、OBJECT ENCODING key、DEBUG OBJECT 等可用于诊断，但生产环境使用危险命令前要评估权限与影响。

3.7 面试追问

问：紧凑编码一定更快吗？

不一定。它主要节省内存并提高缓存局部性，但中间插入、扩容和转换可能需要搬移数据。Redis 依据“小规模时线性操作可接受，大规模时使用通用结构”的原则折中。

问：为什么 key 名过长也会浪费大量内存？

key 本身也是字符串对象，附带对象头、SDS 元数据、哈希表条目和分配器对齐。千万级 key 中每个多 20 字节，累计就可能是数百 MB，还会增加网络与持久化体积。

4. Redis 真的是单线程吗，事件循环与 I/O 线程如何工作？

4.1 结论先行

更准确的说法是：Redis 核心命令执行大多在主线程串行完成，但 Redis 进程并非只有一个线程。后台持久化、异步释放、部分网络 I/O、模块或其他任务可以使用线程或子进程。单线程命令模型简化了数据结构并发控制，但也使慢命令能够阻塞整个实例的命令处理。

4.2 为什么核心命令选择串行执行

- 避免每个数据结构都加细粒度锁。
- 减少线程切换和共享状态竞争。
- 单条命令天然具有“不被其他命令插入”的原子性。
- 内存访问通常很快，很多场景先受网络 RTT 或内存带宽限制。

这并不表示一个实例能线性使用所有 CPU 核。CPU 密集时通常通过分片、多个实例或 Cluster 扩展，而不是无限增加 Redis 命令线程。

4.3 事件驱动模型

Redis 使用多路复用管理大量连接。主循环概念上执行：

1. 等待文件事件或时间事件
2. 读取就绪连接的数据
3. 解析完整命令
4. 执行命令
5. 写入响应或注册可写事件
6. 执行过期、统计等周期任务

一个客户端长时间不发送完整请求不会必然占住一个业务线程，但客户端数量、输入/输出缓冲仍会消耗内存。

4.4 I/O 线程解决什么问题

现代 Redis 可使用 I/O 线程辅助网络读写，把协议数据搬运或 socket 操作并行化；核心命令和共享数据结构操作仍主要由主线程串行完成。I/O 线程适合网络处理成为瓶颈的场景，不能加速一个复杂集合运算或长 Lua 脚本。

调优前应证明瓶颈在网络 I/O，而不是命令 CPU、内存访问、fork、磁盘或客户端。错误地增加 I/O 线程可能增加调度成本，并不能替代分片。

4.5 哪些操作会阻塞主线程

- 对超大集合执行全量遍历、交并集或排序。
- 返回海量数据，如 HGETALL、LRANGE 0 -1、SMEMBERS。

- 删除或过期超大对象，未使用异步释放路径。
- Lua/Function 执行复杂循环。
- KEYS *、不受控的 SCAN 客户端聚合。
- RDB/AOF 后台任务的 fork 阶段造成主线程暂停。
- 内存碎片、缺页、Swap 或宿主机抖动。

4.6 单条命令原子性的边界

INCR counter 对 Redis 数据状态是原子的，但以下业务不是单条命令：

```
GET stock
if stock > 0:
    DECR stock
create database order
```

两个客户端可能同时读到正库存。应使用 Lua/Function 把 Redis 内判断和扣减组合为一次原子执行，再通过消息或数据库约束处理跨系统一致性。

原子执行也不是持久性保证。命令返回成功后，若 AOF 尚未 fsync 或复制尚未到副本，故障时仍可能丢失。

4.7 面试追问

问：既然命令串行，为什么还会出现超卖？

因为超卖通常跨多个命令或跨 Redis 与数据库。每个 GET、DECR 单独原子，不代表“检查 + 扣减 + 下单”整体原子。还要处理重复请求、消息重试和数据库失败补偿。

问：Redis 单机 CPU 100% 怎么办？

先用命令统计、慢日志、CPU profile 和 key 分布定位是热 Key、慢命令、脚本还是后台任务；短期限流/禁用高危命令，长期拆分热 Key、改算法、分片或迁移负载。不能只盲目增加连接和客户端线程。

5. 大 Key、热 Key 和高危命令有什么区别，如何治理？

5.1 概念区分

- **大 Key**：单个 key 的 value 占用内存大，或集合成员数量极多。
- **热 Key**：单个 key 承担的访问 QPS 或带宽过高。
- **高危命令**：复杂度或返回量不可控，可能阻塞实例，例如对大集合执行全量操作。

三者可能重叠，也可能独立。一个 50 字节的全局配置 key 可能是热 Key；一个 100 MB 的冷备份 key 是大 Key 但不热；普通 HGET 访问大 Hash 的单字段可能很快，但 HGETALL 就变成高危操作。

5.2 大 Key 的危害

- 单次读取造成大网络响应和客户端内存峰值。
- 删除、过期、淘汰时可能阻塞或产生大量异步释放工作。
- RDB、AOF rewrite、复制全量同步和 Cluster 迁槽耗时增加。
- 一个集合集中在单槽，无法通过 Cluster 把成员自动拆到多个节点。
- 故障切换后副本加载与恢复时间延长。

大 Key 没有统一字节阈值。需要按实例内存、延迟 SLO、网络 and 对象类型制定标准，例如 String 超过 1 MB 或集合超过十万成员就进入告警；具体阈值应压测验证。

5.3 热 Key 的识别

可组合使用：

- 客户端埋点：按命令、key 前缀、槽统计请求和延迟。
- Redis INFO commandstats、INFO latencystats。
- redis-cli --hotkeys（受淘汰策略和采样条件影响）。
- 代理层、Cluster 节点、网卡和 CPU 不均衡指标。
- 应用日志和链路追踪。

不要在高峰期对全量 key 做高成本精确统计。生产诊断优先采样、前缀聚合和离线扫描。

5.4 大 Key 的治理方案

拆分集合

```
followers:user:1001:00  
followers:user:1001:01  
...  
followers:user:1001:63
```

按 member hash、时间或业务分区拆分。代价是跨分片聚合、分页和原子操作更复杂。

避免整对象读写

用 Hash 字段、分页范围、压缩后的适度块，替代巨大 JSON 每次 GET/SET。压缩能减少网络和内存，但增加 CPU，且不能解决单 key 热点和迁槽问题。

异步删除

使用 UNLINK 或配置惰性释放，让内存回收在后台处理。UNLINK 只降低释放阶段的主线程阻塞，命令仍需从字典摘除 key，且后台积压过多也会占资源。

渐进式扫描

集合使用 HSCAN、SSCAN、ZSCAN 分批处理；注意 SCAN 是游标迭代，不是事务快照，业务必须容忍重复和变化。

5.5 热 Key 的治理方案

- 对只读热点做应用本地缓存，并设置短 TTL、版本号或主动失效。
- 把一个逻辑计数拆成多个分片计数，读取时聚合；接受短暂误差。
- 通过副本读分担只读流量，但必须接受复制延迟和故障切换语义。
- 对不可拆热点做请求合并、SingleFlight、限流和降级。
- Cluster 中调整 key 设计或槽分布，避免多个热点落在同一主节点。

随机后缀复制热点 key 时必须解决更新广播和一致性；不能只写一份、随机读多份旧值。

5.6 高危命令治理

- 生产 ACL 限制 KEYS、FLUSHALL、EVAL、CONFIG、DEBUG 等危险能力。
- 代码评审检查无界范围，例如 LRANGE 0 -1。
- 对集合分页设置最大 page size。
- 脚本设置业务上限，禁止按未知规模循环。
- 建立命令复杂度白名单和慢日志告警。

5.7 面试追问

问：如何无停机拆分一个大 Hash？

创建新分片结构，后台用 HSCAN 分批迁移；写路径在迁移期间双写或按迁移标记路由；读路径先读新结构，未命中回源旧结构并回填；校验数量和摘要后切换，最后异步删除旧 key。必须考虑迁移期间更新覆盖、TTL 继承和失败重试。

6. Key 的过期删除和内存淘汰是如何工作的？

6.1 结论先行

过期与淘汰是两套不同机制：

- **过期**：key 到达 TTL 后应该失效，与内存是否充足无关。
- **淘汰**：实例超过 `maxmemory` 时，根据策略删除某些 key，以便继续写入。

Redis 使用被动过期和主动过期结合，避免为每个 key 创建独立定时器。淘汰策略通常采用采样近似算法，而不是维护全局精确 LRU 列表。

6.2 被动与主动过期

被动过期

客户端访问 key 时发现其已过期，Redis 删除并按不存在处理。优点是只处理真正访问的 key；缺点是永远不再访问的过期 key 会占内存。

主动过期

Redis 周期性从带 TTL 的 key 中采样，删除已过期项，并根据命中比例自适应增加工作。它避免全量扫描，但意味着过期不是毫秒级准时物理删除：语义上 TTL 到期后不可见，内存回收可能稍后发生。

主节点负责形成过期删除并传播给 AOF/副本，以降低各节点独立时钟导致的不一致。TTL 使用绝对时间表达，机器时钟异常会影响过期行为，所以需要可靠时间同步。

6.3 常见淘汰策略

策略	候选范围	选择依据	适用场景
noeviction	不淘汰	写命令返回错误	数据不能被任意删除
allkeys-lru	所有 key	近似最近最少使用	通用缓存
allkeys-lfu	所有 key	近似最不常使用	热度差异明显的缓存
allkeys-random	所有 key	随机	极少使用
volatile-lru	仅有 TTL key	近似 LRU	同实例混合缓存和不可淘汰数据，但风险较高
volatile-lfu	仅有 TTL key	近似 LFU	有 TTL 数据按频率淘汰
volatile-ttl	仅有 TTL key	剩余 TTL 较短优先	TTL 能体现价值时
volatile-random	仅有 TTL key	随机	特殊场景

若使用 `volatile-*` 但没有足够带 TTL 的 key，Redis 可能无法释放足够内存，最终表现类似 `noeviction`。通常建议不同持久性等级的数据使用不同实例，而不是依赖混合策略。

6.4 LRU 与 LFU 的区别

LRU 关注“最近是否访问”；LFU 关注“长期访问频率”，并带时间衰减。突发扫描可能把真正热点挤出 LRU，而 LFU 对稳定热点更友好；但热点变化很快时，LFU 衰减参数也需要调优。

Redis 的 LRU/LFU 是基于对象元数据和候选采样的近似算法，目的是用较低内存和 CPU 获得接近最优的命中率。

6.5 maxmemory 不等于进程 RSS 上限

maxmemory 主要约束数据集相关内存。还需预留：

- 复制 backlog 和客户端输出缓冲。
- AOF 缓冲与 rewrite 增量。
- fork 写时复制额外页。
- jemalloc 碎片和未归还操作系统的内存。
- Lua、模块、线程栈和进程开销。

因此一台 32 GB 主机不能简单设置 maxmemory 31gb。若持久化和复制活跃，常需显著预留，并通过压力测试观察峰值 RSS。

6.6 TTL 设计实践

- TTL 加随机抖动，避免大量 key 同时到期。
- 永久 key 要有明确理由和容量预算。
- 更新 value 时注意某些命令会覆盖或保留 TTL 的差异，可使用 KEeptTL 等能力。
- 缓存 TTL 是新鲜度和数据库保护之间的权衡，不是越短越正确。
- 监控 expired_keys、evicted_keys、命中率和写错误。

6.7 面试追问

问：key 到期后为什么内存没有立即下降？

可能尚未被主动采样、对象由异步线程释放、分配器保留空闲页未归还 OS，或 RSS 包含 fork/COW 和碎片。应同时看逻辑 used_memory、RSS、过期速率和 allocator 指标。

第二部分：原子性、缓存与业务模式

7. Pipeline、事务、Lua 与 Redis Functions 应如何选择？

7.1 结论先行

四者解决的问题不同：

- **Pipeline**：减少网络往返，提高吞吐，不提供原子性。
- **MULTI/EXEC 事务**：把已排队命令连续执行，中间不插入其他客户端命令，但不支持读取前一条结果后动态决定下一条命令。
- **Lua 脚本**：在服务端一次执行带条件逻辑的多命令操作，具有原子执行语义。
- **Redis Functions**：把服务端逻辑作为函数库加载和调用，适合比临时 EVAL 更可管理的脚本能力。

选择顺序通常是：单条原子命令优先；只为减少 RTT 用 Pipeline；固定命令组可用事务；需要读后判断和多步原子修改时用 Lua/Function。

7.2 Pipeline 的工作方式

普通请求每条命令都经历一次 RTT：

```
send SET -> wait reply -> send INCR -> wait reply -> send EXPIRE -> wait reply
```

Pipeline 将多条命令连续发送，再批量读取响应：

```
send SET + INCR + EXPIRE -> read three replies
```

Redis 仍按顺序逐条执行。Pipeline 的性能收益通常来自减少 RTT 和系统调用，而不是让命令并行执行。

7.3 Pipeline 的边界

- 其中一条失败不会自动撤销其他命令。
- 批次太大时，客户端和服务端都需要缓存请求/响应，可能造成内存峰值。
- Cluster 下不同 key 可能路由到不同节点，客户端需要按节点拆分 Pipeline。
- 网络超时后客户端可能不知道服务端执行到哪里；盲目重试非幂等命令会重复写。

生产中应限制批次条数和总字节，常按几百到几千条或几 MB 分批，具体值通过延迟和内存压测确定。

7.4 MULTI/EXEC 的特点

```
MULTI
SET order:1:status CREATED
INCR user:1:order_count
EXEC
```

命令在 MULTI 后进入队列，EXEC 时连续执行。其他客户端命令不会插入事务命令之间。

事务并不提供数据库式隔离视图和回滚。事务内命令排队时无法读取真实执行结果，再基于结果动态分支。语法错误可能导致 EXEC 整体拒绝，运行期类型错误则只影响对应命令，其他命令仍执行。

7.5 Lua 脚本的适用场景

库存扣减：

```

local stock = tonumber(redis.call('GET', KEYS[1]) or '-1')
if stock <= 0 then
    return {0, stock}
end
redis.call('DECR', KEYS[1])
redis.call('SADD', KEYS[2], ARGV[1])
return {1, stock - 1}
    
```

脚本在 Redis 中原子执行，适合“检查 -> 修改多个结构 -> 返回结果”。但脚本运行期间会阻塞其他命令，因此：

- 只操作传入的 key，不做全库扫描。
- 给循环和处理条数设置上限。
- 脚本代码可版本化，返回值协议要稳定。
- Cluster 中脚本涉及的 key 必须满足同槽要求，并通过 KEYS 参数显式传入。

7.6 Functions 相比 EVAL 的价值

EVAL 把脚本随请求发送或通过摘要缓存；Functions 可以把函数库加载到 Redis，按名字调用，便于部署、复用和权限管理。它仍运行在 Redis 服务端，仍要避免长时间阻塞。

面试中不必把 Functions 说成“存储过程完全替代品”。Redis 的职责仍应保持简单、短执行和数据局部性，复杂业务流程应留在应用层。

7.7 选择矩阵

需求	推荐方式
批量读取 500 个独立 key，允许非原子	Pipeline / MGET（同槽和数据量允许时）
两个计数同时增加，不需读结果分支	MULTI/EXEC 或 Lua
检查库存后扣减并记录用户	Lua/Function
单 key 自增并设置 TTL	优先 Redis 8.8 的适用原子命令；跨版本可 Lua
批量导入数万条数据	分批 Pipeline，控制背压
跨 Redis 与数据库原子提交	四者都不能直接解决，需要业务一致性方案

7.8 面试追问

问：Lua 脚本执行超时后能否安全重试？

不能仅凭客户端超时判断脚本未执行。脚本可能已执行成功但响应丢失。调用应设计业务幂等标识，或者返回可查询的执行结果；重试前先确认状态。

8. Redis 事务为什么不支持传统回滚，WATCH 能解决什么？

8.1 结论先行

Redis 事务的核心保证是命令组在 EXEC 后按顺序连续执行，而不是关系数据库的 ACID 事务。Redis 不对运行期错误自动回滚，因为其命令通常是确定性的内存操作，避免维护 undo 日志可以保持实现简单和高性能。

WATCH 提供乐观锁/CAS：如果被监视的 key 在 EXEC 前发生变化，事务不执行，客户端重新读取并重试。

8.2 两类错误

入队前可发现的错误

命令名错误、参数数量错误等会使事务在 EXEC 时拒绝执行。

执行期错误

例如对 String 执行 List 命令：

```
MULTI
SET a 1
LPUSH a x
SET b 2
EXEC
```

SET a 和 SET b 仍可能成功，LPUSH 返回类型错误。Redis 不回滚已成功命令。

8.3 WATCH 的工作模式

```
WATCH account:1
GET account:1
# 客户端计算新值
MULTI
SET account:1 newValue
EXEC
```

若从 WATCH 到 EXEC 期间其他客户端修改了 key，EXEC 返回失败/空结果，客户端需要重试。它适合冲突率低、计算逻辑在客户端的 CAS 更新。

8.4 WATCH 的问题

- 高冲突下反复重试，吞吐下降。
- 从读取到 EXEC 有多次 RTT。
- 客户端必须正确处理超时、连接中断和 UNWATCH。
- Cluster 下所有相关 key 必须同槽。
- 只解决 Redis 内部并发，不解决数据库或 MQ 的跨系统一致性。

对高并发库存扣减，Lua 通常比 WATCH 更合适，因为逻辑在服务端一次完成，减少冲突重试和 RTT。

8.5 Redis 事务与数据库事务对比

维度	Redis MULTI/EXEC	关系数据库事务
原子执行	命令组不被插入，但运行期错误不回滚	通常支持整体提交/回滚
隔离	没有 MVCC 查询快照	提供多种隔离级别
一致性约束	由命令和应用保证	唯一、外键、检查约束等
持久性	取决于 RDB/AOF 与 fsync	取决于 WAL/redo 与配置
乐观锁	WATCH	版本号、条件更新等

8.6 高质量回答

Redis 事务不是缩小版数据库事务。它主要提供命令队列和连续执行，执行期错误不回滚。需要 CAS 时可用 WATCH，但高冲突业务更适合单条原子命令或短 Lua。跨 Redis 和数据库仍要靠唯一约束、状态机、消息或补偿，而不能靠 MULTI/EXEC。

9. 如何设计缓存读写模式并处理数据库一致性？

9.1 结论先行

最常用的是 Cache Aside：应用读缓存，未命中读数据库并回填；写时先更新数据库，再删除缓存。它追求最终一致性而非强一致，必须通过 TTL、幂等删除、消息重试、版本校验和数据库兜底控制不一致窗口。

不存在对所有故障都完美的“先后顺序”。先更新库再删缓存是常见默认选择，因为先删缓存再更新库更容易被并发读写回旧值。

9.2 Cache Aside 读路径

```
1. GET cache
2. hit -> return
3. miss -> query DB
4. SET cache with TTL
5. return
```

需要防止多个并发 miss 同时打到数据库，可用请求合并、本地 SingleFlight、短锁或逻辑过期异步刷新。

9.3 写路径：先数据库，后删除缓存

```
BEGIN DB TX
  UPDATE product SET ...
COMMIT
DEL cacheKey
```

选择删除而不是直接更新缓存的原因：

- 缓存可能是多表聚合，写路径难以准确重建。
- 更新缓存时可能与并发写发生乱序覆盖。
- 删除后由下一次读按当前数据库状态重建，更简单。

仍有失败窗口：数据库提交成功后应用宕机，缓存未删除。可通过重试、事务消息、Outbox、binlog/CDC 订阅删除和 TTL 收敛。

9.4 为什么“先删缓存，再更新数据库”危险

时序：

```
Writer: DEL cache
Reader: cache miss -> read old DB value
Writer: UPDATE DB commit
Reader: SET old value into cache
```

旧值会持续到 TTL 或下一次删除。延迟双删在更新后再延迟删除一次可降低部分概率，但延迟值难以覆盖所有慢查询和调度，不能替代可靠事件和版本校验。

9.5 删除缓存也可能乱序

两个写请求：

```
W1 update DB v1
W2 update DB v2
W2 delete cache
Reader load v2 and set cache
W1 delayed delete cache
```

结果只是缓存被多删一次，下一次重新加载 v2，通常安全。若是“写库后直接 SET 缓存”，W1 延迟写缓存则可能把 v1 覆盖 v2。

9.6 可靠删除方案

本地重试 + 失败队列

删除失败按指数退避重试，最终进入死信或人工队列。删除是幂等操作，适合重试。

Transactional Outbox

数据库事务同时写业务表和 outbox 事件，后台发布事件删除缓存。避免“数据库成功、事件没记录”，但事件仍需幂等消费和清理。

Binlog/CDC

订阅数据库变更生成缓存失效事件。优点是业务代码侵入小；缺点是映射关系、延迟、回放和运维更复杂。

版本化缓存

缓存 value 包含数据库版本或更新时间。回填时使用 Lua 或条件写，只允许新版本覆盖旧版本，防止慢请求把旧值写回。

9.7 强一致需求怎么办

若业务绝不能读旧值：

- 不缓存该数据，直接访问权威数据库。
- 在写后短时间内走主库或携带版本读。
- 使用租约、读写串行化或一致性服务，但成本和可用性下降。
- 把缓存只作为性能提示，最终操作仍以数据库条件更新和约束判定。

账户余额、库存最终扣减等不能仅凭缓存值决定事实。

9.8 面试追问

问：缓存删除成功，数据库事务回滚怎么办？

只是额外 miss，下一次从旧数据库状态回填，不会把错误新值写入。因此先提交数据库再删除缓存更常见。

问：读请求回填时数据库刚被更新，如何避免旧值写入？

使用版本号比较、删除事件在回填后再次触发、或让写路径和回填通过短锁/SingleFlight 协调。若窗口不可接受，应重新评估是否适合缓存。

10. 缓存穿透、击穿、雪崩和缓存污染如何治理？

10.1 四个概念

- **穿透**：请求的数据本来就不存在，缓存和数据库都 miss。
- **击穿**：单个热点 key 失效，大量并发同时回源。
- **雪崩**：大量 key 同时失效或整个缓存服务不可用，数据库被洪峰压垮。
- **缓存污染**：低价值或一次性数据挤占内存，淘汰真正热点。

治理不能只在 Redis 层，应覆盖入口限流、应用请求合并、缓存策略、数据库保护和降级。

10.2 缓存穿透

参数校验和权限校验

非法 ID、越权请求在访问缓存前拒绝。不要让攻击者构造无限随机 key。

缓存空值

数据库确认不存在后缓存短 TTL 的空标记：

```
product:invalidId -> NULL_MARKER, TTL 30s
```

需要防止真实数据随后创建而空值未失效，写路径应删除空缓存。

Bloom Filter

在请求前判断 key 是否“可能存在”。不存在则直接拒绝；可能存在仍需访问缓存/数据库。Bloom 有假阳性但无假阴性（前提是正确维护），适合数据集相对稳定、规模大且可容忍误判回源。

删除频繁时普通 Bloom 不方便，可使用 Cuckoo Filter 或重建/分层策略。

10.3 缓存击穿

互斥重建

缓存 miss 后只有一个请求获取短锁并查询数据库，其他请求等待、返回旧值或降级。锁要有 TTL、唯一持有者和超时处理。

逻辑过期

value 中保存逻辑过期时间，物理 key 不立刻删除：

```
{"expireAt": 1780000000, "data": {...}}
```

过期后返回旧值，并异步由一个线程刷新。适合允许短暂旧数据的热点场景，避免请求阻塞；需要控制最长陈旧时间和刷新失败告警。

提前刷新和请求合并

高热 key 在 TTL 前异步续期，应用内使用 SingleFlight 合并相同 key 的并发加载。

10.4 缓存雪崩

- TTL 加随机抖动，按业务批次错峰。
- Redis 做主从/Cluster，高可用切换要经过演练。
- 应用设置超时、熔断、限流和线程池隔离。
- 数据库限流和只读降级，避免缓存故障拖死主库。
- 多级缓存：本地缓存保留极热点和配置快照。
- 预热时分批加载，不在发布瞬间灌入全部数据。

高可用不等于不会雪崩。错误 ACL、全量 Flush、客户端连接风暴、证书过期和网络故障都可能让所有节点“逻辑不可用”。

10.5 缓存污染

- 选择 allkeys-lfu 等更符合热度的策略。
- 一次性查询不写入共享缓存，或设置很短 TTL。
- 大对象、报表和分页深页使用独立缓存空间。
- 监控命中率不能只看全局平均，应按业务和 key 前缀分层。

10.6 保护链路

```
Gateway rate limit
-> application concurrency limit
-> local cache / request coalescing
-> Redis timeout + circuit breaker
-> DB connection pool bulkhead
-> fallback / stale value / friendly error
```

一个完整方案必须说明：当 Redis 超时、数据库变慢和重建失败时，系统如何保持可用，而不是只说“加分布式锁”。

11. 如何实现一个正确的 Redis 分布式锁？

11.1 结论先行

单实例 Redis 锁的基本安全写法是：用 `SET key randomValue NX PX ttl` 原子加锁，解锁时用 Lua 比较 value 后删除。它能避免误删他人锁，但仍存在租约过期后旧持有者继续执行的问题。对会修改外部资源的严格场景，需要 fencing token、数据库条件更新或真正的共识锁服务。

分布式锁不是业务幂等的替代品，也不应被用来掩盖错误的数据模型。

11.2 错误实现

```
SETNX lock:order 1
EXPIRE lock:order 30
```

若进程在两条命令之间宕机，锁没有 TTL，可能永久死锁。应使用单条 SET：

```
SET lock:order 9f8c... NX PX 30000
```

value 必须是每次加锁唯一的随机值或请求标识。

11.3 安全解锁

错误：

```
DEL lock:order
```

假设客户端 A 锁过期，B 获得同名新锁，A 业务完成后执行 DEL，会误删 B 的锁。

正确做法：

```
if redis.call('GET', KEYS[1]) == ARGV[1] then
    return redis.call('DEL', KEYS[1])
else
    return 0
end
```

比较和删除必须原子执行。

11.4 锁续期与看门狗

若业务时长不确定，客户端可定期续期，但续期时也必须校验持有者。Redisson 看门狗会在客户端仍存活且持锁时自动续期。

续期不能解决所有问题：

- JVM 长时间 STW、容器暂停或网络隔离可能让续期错过。
- 客户端恢复后可能继续执行，但锁已被他人获得。

- 外部系统无法知道请求是否仍有锁资格。

因此需要 fencing token。

11.5 Fencing Token

锁服务每次成功授予锁时返回单调递增 token：

```
A gets token 41
A pauses, lock expires
B gets token 42 and writes resource
A resumes and tries token 41
resource rejects because 41 < latest 42
```

下游数据库/存储需要保存最后 token，并通过条件更新拒绝旧 token：

```
UPDATE resource
SET value=?, fence_token=42
WHERE id=? AND fence_token < 42;
```

这把“租约过期后旧持有者继续写”的问题交给真正被保护的资源判断。

11.6 主从切换的安全窗口

Redis 主从复制通常异步：

```
A 在主节点加锁成功
锁尚未复制到副本
主节点宕机
副本提升
B 在新主节点获得同一把锁
```

A 和 B 可能同时认为持锁。**WAIT** 可让写等待一定数量副本确认，缩小窗口，但不是强一致共识，也不能消除所有分区和故障语义。

11.7 Redlock 争议应如何回答

官方文档描述了在多个独立 Redis 实例上获取多数锁的 Redlock 算法。业界争议集中在时钟、进程暂停、网络延迟和租约语义。高质量回答不应简单说“Redlock 绝对安全”或“完全不能用”，而应区分风险等级：

- 防止重复定时任务、降低并发冲突：合理的 Redis 租约锁通常足够。
- 资金、主节点选举、不可重复外部副作用：应使用数据库唯一约束、fencing token、ZooKeeper/etcd 等共识组件或业务状态机。

11.8 锁的工程清单

- 锁 key 粒度合理，避免全局热点。
- value 唯一，原子加锁，比较后解锁。
- TTL 大于预期业务时间，并有最大执行时间。

- 超时和重试带随机退避，防止惊群。
- 业务操作幂等，下游有唯一约束。
- 记录获得锁、等待、续期失败和持锁时长指标。
- 明确 Redis 故障时是拒绝业务还是无锁降级。

12. Redis 限流器应如何设计？

12.1 结论先行

常见限流算法有固定窗口、滑动窗口日志、滑动窗口计数、令牌桶和漏桶。Redis 适合通过原子计数、Sorted Set 或 Lua 实现分布式限流，但要同时考虑时间来源、热 Key、内存、Cluster 同槽、超时降级和多级限流。

12.2 固定窗口

```
key = rate:user:1001:202606291230
INCR key
EXPIRE key 120
```

窗口内计数超过阈值则拒绝。实现简单、内存小，但边界突刺明显：前一分钟最后一秒和下一分钟第一秒可通过两倍流量。

INCR 和首次 EXPIRE 应在 Lua 中原子组合，避免计数 key 无 TTL。Redis 8.8 新增的窗口计数相关命令可在支持版本中简化实现，但跨版本面试答案仍应掌握 Lua 方案。

12.3 滑动窗口日志

每个请求以时间戳/唯一 ID 写入 ZSet：

```
local now = tonumber(ARGV[1])
local window = tonumber(ARGV[2])
local limit = tonumber(ARGV[3])
redis.call('ZREMRANGEBYSCORE', KEYS[1], 0, now-window)
local count = redis.call('ZCARD', KEYS[1])
if count >= limit then return 0 end
redis.call('ZADD', KEYS[1], now, ARGV[4])
redis.call('PEXPIRE', KEYS[1], window)
return 1
```

精确但每个请求保存一个 member，热点用户和高 QPS 会消耗大量内存。member 不能只用毫秒时间戳，否则同毫秒请求可能覆盖，应加入随机/序列。

12.4 滑动窗口计数

把时间切为更小桶，例如每 10 秒一个计数，聚合最近 6 个桶。内存比日志小，但有近似误差。适合网关高吞吐限流。

12.5 令牌桶

状态包括令牌余额和上次补充时间。每次请求根据时间差补充令牌，再尝试扣减。支持平均速率和突发容量，适合 API 限流。

所有计算应在 Lua/Function 中完成。使用客户端时间可能被时钟漂移影响，可使用 Redis `TIME`，但在 Cluster 和跨区域场景仍需评估时钟及路由。

12.6 多级限流

- 网关：租户、IP、接口粗粒度限流。
- 应用本地：保护线程池和依赖，避免每次都访问 Redis。
- Redis 全局：多实例共享额度。
- 下游：数据库连接池、第三方 API 独立隔离。

若 Redis 限流服务不可用，要明确 fail-open 还是 fail-closed：营销页面可开放并依赖本地保护，资金接口可能必须拒绝。

12.7 热 Key 优化

全站一个限流 key 会成为热点。可按租户、用户、接口拆分；全局额度可分配给多个实例做本地配额并周期校正，牺牲小幅精度换吞吐。严格全局限流往往需要接受单点协调成本。

13. 排行榜、计数器、延时任务和滑动窗口如何实现？

13.1 排行榜

用 ZSet: member 是用户, score 是积分。

```
ZINCRBY rank:season:2026 10 user:1001
ZREVRANK rank:season:2026 user:1001
ZRANGE rank:season:2026 0 99 REV WITHSCORES
```

生产设计需考虑:

- 同分如何排序: 把时间或序列编码到 member, 或维护第二维规则。
- 周榜/月榜: 按周期独立 key, 设置 TTL。
- 大榜单分页: 使用 score/member 游标, 不要深 offset。
- 更新与数据库账本: Redis 做实时榜, 数据库保存积分流水并可重建。

13.2 分片计数器

超热计数器可拆为 N 个:

```
page:view:123:0 ... page:view:123:63
```

写入随机或按请求 hash 选择分片, 读取时 MGET 聚合。降低单 key 热点, 但读放大、精确实时值和重置更复杂。对展示型计数可异步聚合, 对资金计数不应采用这种近似方案。

13.3 延时任务

用 ZSet 的 score 保存执行时间:

```
ZADD delay:order 1780000123456 order:1001
```

消费者查询到期任务并抢占。错误方案是 ZRANGEBYSCORE 后 ZREM 分两步, 多个消费者会重复获取。应使用 Lua 原子弹出, 或新版本支持的适合命令; 仍要处理消费者取走后宕机, 因此需要 processing 集合、可见性超时和重试。

专业延时消息系统在可靠性、持久堆积和运维方面更成熟。ZSet 方案适合规模可控、允许应用补偿的任务。

13.4 滑动窗口统计

将事件写入 ZSet, score 为时间, member 唯一; 删除窗口外数据后统计。适合“最近 5 分钟失败次数”。

风险: 高 QPS 下单 key 成为大 Key; 每次删除和计数成本增加。可以按分钟桶计数、时间分片 key 或使用时序系统。

13.5 去重与幂等

```
SET idem:payment:requestId result NX EX 86400
```

仅 SET NX 记录“处理中”时，要处理业务失败是否删除、进程宕机后状态恢复、结果查询和 TTL 是否覆盖重试周期。更稳健的状态：

```
PROCESSING -> SUCCESS(result) / FAILED(retryable)
```

支付等核心幂等最终还需数据库唯一索引，因为 Redis 数据可能过期、丢失或切换。

14. Bitmap、HyperLogLog、Bloom Filter、Top-K 如何选型？

14.1 结论先行

这些结构用空间效率换功能或精度：

- Bitmap：精确保存布尔状态，适合稠密整数 ID。
- HyperLogLog：近似基数统计，只回答“有多少不同元素”。
- Bloom Filter：判断“肯定不存在 / 可能存在”，有假阳性。
- Top-K：近似找高频元素，不保证完整精确排名。

不能把概率结构用于资金、库存或必须可审计的精确事实。

14.2 Bitmap

用户 ID 作为 offset：

```
SETBIT signin:2026-06-29 1001 1
GETBIT signin:2026-06-29 1001
BITCOUNT signin:2026-06-29
```

一个 bit 表示一个状态，空间非常小。问题在于最大 offset 决定字符串长度：若用户 ID 稀疏且最大值巨大，会分配大空间。可先做 ID 映射或分段位图。

可用 BITOP 做多日活跃交并，但超大位图操作会消耗 CPU，并产生大结果。

14.3 HyperLogLog

```
PFADD uv:2026-06-29 user1 user2 user3
PFCOUNT uv:2026-06-29
```

固定较小空间估算去重数，典型误差约百分之一量级，适合 UV 和大规模基数趋势。不能枚举用户，也不能判断某个用户是否已出现。

需要精确去重且集合规模可控时用 Set；需要趋势估计且规模巨大时用 HLL。

14.4 Bloom Filter

初始化时需估计容量和错误率。错误率越低、容量越大，所需内存越多。使用流程：

```
BF.RESERVE products 0.001 10000000
BF.ADD products product:1001
BF.EXISTS products product:9999
```

返回不存在时可直接判定不存在；返回存在仍可能是假阳性，需要继续查询。容量超过设计值后误判率会恶化，需监控或使用可扩展配置。

14.5 Top-K 与 Count-Min Sketch

Top-K 用于找高频 IP、搜索词、商品；Count-Min Sketch 估算某元素频率。二者适合流式监控和候选发现，不适合精确计费。

在 Redis 8 统一发行版中，概率数据结构已成为官方统一能力的一部分，但仍应确认部署版本、命令可用性和持久化/复制兼容性。

14.6 选择表

问题	结构
用户今天是否签到	Bitmap
今天大约有多少独立访客	HyperLogLog
商品 ID 一定不存在吗	Bloom Filter
最近流量中最热的 100 个 IP	Top-K
用户是否属于某个精确集合	Set
精确排行榜	Sorted Set

15. Pub/Sub、Streams 与专业消息队列有什么差异？

15.1 结论先行

Redis Pub/Sub 是实时广播：订阅者在线才能收到，消息不持久、无 ACK。Streams 是持久追加日志，支持消费者组、待确认列表和重放。专业 MQ 通常在分区扩展、长期堆积、复制语义、管理生态和跨地域方面更完整。

15.2 Pub/Sub

```
SUBSCRIBE config-change
PUBLISH config-change product:1001
```

适合在线通知、聊天室、配置刷新等“错过可接受”场景。订阅者断线期间的消息不会补发。慢消费者还可能造成输出缓冲增长并被断开。

Redis Cluster 中还需区分全局 Pub/Sub 与 Sharded Pub/Sub；后者按槽传播，减少全集群广播开销。

15.3 Streams

Stream 条目带单调 ID：

```
XADD orders * orderId 1001 userId 88
XREAD COUNT 10 BLOCK 5000 STREAMS orders $
```

消费者组将消息分配给组内消费者：

```
XGROUP CREATE orders order-workers 0 MKSTREAM
XREADGROUP GROUP order-workers c1 COUNT 10 BLOCK 5000 STREAMS orders >
XACK orders order-workers 17800000000000-0
```

未 ACK 的消息进入 PEL，可查看、认领和重试。

15.4 与 Kafka/RabbitMQ 的差异

维度	Pub/Sub	Streams	Kafka/RabbitMQ 等
离线补发	否	是	是
ACK/消费状态	否	消费组 PEL	成熟
长期海量堆积	不适合	规模受 Redis 内存/持久化限制	通常更适合
顺序	单连接接收顺序	Stream ID 顺序，组内并行需谨慎	分区/队列语义明确
运维工具	简单	中等	完整生态
主要价值	低延迟广播	轻量持久事件流	可靠消息平台

15.5 决策原则

- 只需在线通知，丢失可接受：Pub/Sub。
- 已有 Redis、流量可控、需要轻量消费组：Streams。

- 事件是核心业务资产、需长期回放和大规模堆积：专业 MQ。

不要只比较“能否发送消息”，还要比较恢复、积压、监控、跨机房和成本。

16. Redis Streams 消费组、ACK、PEL、重试和幂等如何设计？

16.1 消费组模型

同一组内消息通常分配给一个消费者，多个组可独立消费同一 Stream。新消息读取使用 `>`；指定历史 ID 可读取已分配消息。

处理链路：

```
XREADGROUP -> business process -> durable commit -> XACK
```

ACK 应在业务结果可靠落地之后。如果先 ACK 再写数据库，进程宕机会丢业务；先写数据库后 ACK，ACK 丢失会重复消费，因此消费逻辑必须幂等。

16.2 PEL (Pending Entries List)

PEL 记录已投递但未 ACK 的消息、消费者、空闲时间和投递次数。监控：

- PEL 总量是否持续增长。
- 最老 pending 的 idle 时间。
- 消费者是否长期无心跳。
- 重试次数和死信数量。

消费者宕机后，可用 `XAUTOCLAIM` 等机制把长时间未确认消息转移给其他消费者。Redis 8.8 的 `XNACK` 提供显式释放 pending 的新能力，但兼容设计仍应掌握 claim/retry 流程。

16.3 幂等消费

数据库唯一约束

```
INSERT INTO processed_event(event_id, ...)
VALUES (?, ...)
ON CONFLICT / ON DUPLICATE KEY ...;
```

事件 ID 唯一，业务写和幂等记录放同一数据库事务中，是核心业务最可靠的方式之一。

Redis 幂等标记

```
SET processed:eventId 1 NX EX 7d
```

适合非核心、重试窗口有限场景。必须考虑标记成功但业务失败，或业务成功但标记丢失的问题；通常需要状态机或 Lua，但跨数据库仍无法原子。

16.4 重试与死信

- 短暂错误：指数退避，不要立即高频重试。

- 永久错误：超过次数写入死信 Stream，告警并提供人工重放。
- 业务限流：避免积压恢复时瞬间压垮数据库。
- 毒消息：记录错误原因和原始载荷，禁止无限循环。

Streams 本身不会替应用判断错误类型。消费者需要明确可重试、不可重试和补偿语义。

16.5 Stream 长度与裁剪

```
XADD orders MAXLEN ~ 1000000 * ...  
XTRIM orders MAXLEN ~ 1000000
```

近似裁剪通常性能更好。裁剪前要确认消费者不需要更早历史，且 PEL 中消息的处理语义不会被破坏。对于审计型事件，应先归档到数据库或对象存储。

16.6 一致性边界

Redis Streams 与数据库之间没有分布式事务。常见方案：

- 数据库 Outbox -> 发布到 Stream。
- Stream 消费 -> 数据库幂等事务 -> ACK。
- 定期对账补发。

若事件不能丢，不能只依赖应用“先写 DB 再 XADD”且没有补偿，因为两步之间宕机会缺事件。

第三部分：持久化、复制与高可用

17. RDB 与 AOF 的机制、取舍和恢复顺序是什么？

17.1 结论先行

Redis 提供两类核心持久化：

- **RDB**：某一时刻的数据快照，文件紧凑、恢复快，但两次快照之间的数据可能丢失。
- **AOF**：记录写命令，按 fsync 策略持久化，通常数据丢失窗口更小，但文件更大、写放大和重写成本更高。

生产中常同时启用，利用 RDB 做快速备份和全量恢复，AOF 提供更小的 RPO。持久化不是备份的替代品：误删除或逻辑错误也会被持久化和复制，仍需异地备份与恢复演练。

17.2 RDB 的生成

BGSAVE 时主进程 fork 子进程，子进程遍历内存数据并写临时 RDB，完成后原子替换旧文件。主进程继续服务请求，依赖写时复制维持子进程看到的快照。

优点：

- 文件通常紧凑，适合备份和传输。
- 加载时直接恢复数据结构，速度通常较快。
- 正常运行时额外写盘频率较低。

缺点：

- 最近一次成功快照后的数据在宕机时可能丢失。
- fork 和 COW 在大内存、高写入场景会引起延迟与内存峰值。
- 快照间隔过短增加资源消耗，过长扩大 RPO。

17.3 AOF 的写入链路

写命令执行后追加到 AOF 缓冲，再写入操作系统页缓存，并按策略 fsync：

- **appendfsync always**：每次写事务都尽快 fsync，数据安全更高、延迟更大。
- **appendfsync everysec**：通常每秒 fsync，性能与 RPO 的常用折中，机器故障可能丢失约 1 秒附近的数据。
- **appendfsync no**：交给操作系统决定，吞吐高但丢失窗口不可控。

“每秒 fsync”不意味着任何情况下严格只丢一秒：磁盘阻塞、内核调度和配置会影响实际窗口。应通过监控 AOF 延迟和故障演练验证。

17.4 AOF 重写

AOF 长期记录历史命令会膨胀。重写根据当前数据状态生成等价且更短的新 AOF，例如一千次 INCR 可被压缩为当前值的设置命令。

重写期间：

```
child: 根据 fork 时快照生成基础 AOF
parent: 继续处理写入，并记录重写期间增量
finish: 合并/切换到新 AOF
```

现代 Redis 使用多部分 AOF（基础文件、增量文件、manifest）降低管理复杂度。不要依赖旧版本“单个 appendonly.aof 文件”的固定布局编写备份脚本。

17.5 RDB 与 AOF 同时启用时如何加载

当 AOF 启用且文件完整时，Redis 启动通常优先使用 AOF 恢复，因为 AOF 往往包含更新的数据状态。具体行为应结合版本和 AOF manifest 检查。

如果 AOF 尾部因非正常关机出现不完整写入，Redis 可按配置处理或使用 redis-check-aof 修复。修复可能截断尾部数据，操作前必须备份原文件并评估 RPO。

17.6 持久化配置选择

纯缓存

数据可从数据库完全重建，可关闭持久化以减少磁盘和 fork 开销，但必须保证缓存冷启动、限流和数据库保护。高峰时全量冷缓存可能比开启持久化更危险。

可重建但重建昂贵

启用 RDB，定期外部备份；故障后快速恢复大部分热数据，剩余回源。

Redis 承担重要临时状态

RDB + AOF everysec，配合副本和备份；仍需说明最多可接受的数据丢失窗口。

不允许丢失核心事实

不要仅依赖 Redis。以数据库、消息日志或共识系统为权威，Redis 做派生状态。

17.7 备份与恢复

可靠方案包括：

- 将 RDB/AOF 复制到独立存储和异地账户。
- 保留多时间点版本，防止逻辑错误覆盖所有副本。

- 对备份做校验和、加密和访问控制。
- 定期在隔离环境恢复，测量 RTO 和数据完整性。
- 记录 Redis 版本、配置、模块和 ACL，确保可重建环境。

17.8 面试追问

问：开启 AOF always 是否就不会丢数据？

本地机器故障的数据丢失窗口很小，但仍可能有硬件、文件系统、配置和复制切换问题。主节点写成功后副本未同步就切换，也可能丢失。更重要的是，误操作会被可靠持久化，不能替代备份。

18. fork、写时复制和持久化为何可能造成延迟与内存峰值？

18.1 结论先行

RDB 快照和 AOF 重写常通过 fork 创建子进程。fork 不会立刻复制全部数据页，但需要复制页表，并采用 Copy-on-Write：父子共享物理页，任一方写入时才复制。数据集越大，fork 页表开销越高；写入越活跃，COW 额外内存越大。

这解释了“Redis 数据只有 20 GB，BGSAVE 时 RSS 突然逼近 40 GB”以及“后台保存却出现主线程延迟”。

18.2 fork 阶段为何阻塞

fork 系统调用在主线程执行，需要创建子进程并复制内存页表。大内存实例、透明大页、宿主机压力和页表巨大时，fork 可产生毫秒到秒级暂停。

可观察：

- INFO stats 中 latest_fork_usec。
- LATENCY DOCTOR 的 fork 相关事件。
- 操作系统进程停顿和延迟曲线。

18.3 COW 内存峰值

fork 后：

```
parent and child share page P
parent modifies key in P
OS allocates new page P'
parent writes P', child keeps P
```

高写入率、修改分布广时，大量页被复制。不是“修改多少业务字节就复制多少字节”，通常按内存页复制，一个小字段更新也可能触发整个页复制。

需要预留足够系统内存。否则触发 Swap、OOM Killer 或 fork 失败，造成持久化失败和延迟雪崩。

18.4 磁盘和内存共同影响

- 子进程写 RDB/AOF 占用磁盘带宽，影响 AOF fsync。
- 页缓存竞争可能挤出 Redis 热页，引发缺页。
- 云盘突发额度耗尽，rewrite 时间显著延长。
- AOF rewrite 增量缓冲增长，进一步占内存。

持久化文件不应放在高延迟网络文件系统。备份上传应在文件生成后异步进行。

18.5 常见优化

- 控制单实例内存，不把 200 GB 数据放一个实例；通过分片降低 fork 单元。
- 预留内存，禁用 Swap 或设置严格保护，合理配置 overcommit。
- 检查 Transparent Huge Pages 对延迟/COW 的影响，按 Redis 官方和操作系统建议配置。
- RDB 与 AOF rewrite 错峰，避免与宿主机备份、压缩任务冲突。
- 使用高性能本地 SSD，监控 fsync 和磁盘队列。
- 降低无必要高频写，拆分高写热点。

18.6 为什么不能简单关闭所有持久化

关闭持久化能减少 fork 和磁盘压力，但冷启动会让所有缓存同时回源，可能压垮数据库。应基于业务 RTO/RPO、重建速度和依赖承载能力选择，而不是仅为降低 P99 关闭。

19. Redis 主从复制、全量同步和部分同步如何工作？

19.1 结论先行

Redis 复制通常是主节点向副本异步传播写命令。首次建立复制或无法续接时进行全量同步；短暂断线且复制 backlog 仍保留缺失数据时，可通过 PSYNC 部分同步。异步复制带来低延迟，也意味着主节点确认的写可能在副本接收前丢失。

19.2 全量同步流程

简化流程：

```
replica -> PSYNC
master decides full resync
master BGSAVE / diskless snapshot
master buffers new writes
snapshot transferred to replica
replica clears old data and loads snapshot
master sends buffered incremental writes
replica catches up
```

全量同步成本包括 fork、快照、网络带宽、副本加载时的内存和 CPU。多个副本同时重连可能造成 replication storm。

19.3 部分同步

主节点维护 replication ID、offset 和环形 backlog。副本断线重连时带上已处理 offset：

- offset 仍在 backlog：只发送缺失增量。
- backlog 已覆盖或复制 ID 不匹配：全量同步。

repl-backlog-size 应根据写入字节率和期望断线容忍时间估算：

```
backlog >= peak_replication_bytes_per_sec * reconnect_window * safety_factor
```

只按 QPS 估算不准确，因为写命令和 value 大小差异很大。

19.4 副本读的语义

副本可以分担读，但可能读到旧数据：

- 正常复制延迟。
- 网络抖动和主节点高写。
- 副本正在全量加载。
- 故障切换和拓扑变化。

“读写分离”必须按业务分类：商品详情可接受秒级旧值，刚修改密码后的验证、库存确认和读己之写通常应走主节点或带版本确认。

19.5 WAIT 能保证什么

`WAIT numreplicas timeout` 让客户端等待此前写入被一定数量副本确认收到。它可以降低主节点立即故障时的数据丢失概率，但不是共识提交：

- 确认可能只代表副本接收，不代表副本磁盘持久化。
- 网络分区和故障选择仍可能丢失。
- 增加写延迟和可用性成本。

应表述为“增强复制确认”，不是“把 Redis 变成强一致数据库”。

19.6 主节点与副本的 `maxmemory`

副本通常不独立执行与主节点不同的普通 key 淘汰，以保持数据集一致；主节点淘汰后会传播删除。副本自身仍有复制和客户端缓冲等内存开销。把副本 `maxmemory` 设置得和主节点数据上限完全相同可能不够，应预留复制缓冲。

19.7 复制监控

- 主副本 link 状态和断线时长。
- `master_repl_offset` 与副本 `offset` 差值。
- backlog 大小和历史断线是否触发全量同步。
- 全量同步次数、耗时和网络吞吐。
- 副本加载状态、只读状态。
- `rejected_connections`、输出缓冲和内存峰值。

20. Sentinel 如何完成故障判断、选主与切换？

20.1 结论先行

Sentinel 为非 Cluster 的主从 Redis 提供监控、故障通知、主节点自动切换和服务发现。多个 Sentinel 先各自做主观下线判断，再通过 quorum 达成客观下线；其中一个 Sentinel 获得多数授权成为 leader，选择副本提升并重配置其他节点。

Sentinel 提升可用性，但复制仍通常是异步的，不能保证零数据丢失，也不能解决数据分片。

20.2 主观下线与客观下线

- **SDOWN**：单个 Sentinel 在 `down-after-milliseconds` 内无法获得有效响应，主观认为节点下线。
- **ODOWN**：足够数量 Sentinel 认同主节点下线，达到配置 quorum。

quorum 用于判断故障；真正执行 failover 还需要 Sentinel leader 获得多数 Sentinel 授权。部署 3 个 Sentinel 是常见最小方案，应分散到独立故障域，而不是和 Redis 全放同一台主机。

20.3 选举与故障转移

简化步骤：

1. 多个 Sentinel 判定主节点 ODOWN。
2. Sentinel 之间选出本轮 failover leader。
3. leader 从副本中选择候选。
4. 向候选发送提升命令，使其成为新主。
5. 其他副本改为复制新主。
6. 旧主恢复后被配置为新主的副本。
7. 客户端通过 Sentinel 获取新地址并重连。

20.4 副本选择因素

候选通常考虑：

- 是否长期断线或状态不健康。
- 配置的 replica priority。
- 复制 offset，尽量选择数据更新的副本。
- run ID 等稳定排序因素。

replica-priority 0 可使副本不参与提升，适合远端灾备或特殊节点。

20.5 数据丢失与脑裂窗口

旧主可能在网络分区的少数侧继续接受写，另一侧 Sentinel 把副本提升为新主。旧主恢复后改为副本，它在分区期间接受的写会丢失。

可使用：

```
min-replicas-to-write
min-replicas-max-lag
```

让主节点在没有足够健康副本时拒绝写，缩小脑裂写入窗口，但会牺牲可用性，且不是严格共识。

20.6 客户端必须支持 Sentinel

客户端不能只缓存初始主节点 IP。需要：

- 从多个 Sentinel 查询当前主节点。
- 连接失败后刷新拓扑。
- 处理切换期间短暂超时、READONLY/连接断开。
- 对非幂等写谨慎重试。
- 设置合理 DNS、连接、命令和总请求超时。

Sentinel 完成服务端切换不代表应用零感知。连接池中的旧连接需要淘汰，切换时间内业务应限流和降级。

20.7 Sentinel 与 Cluster 的区别

维度	Sentinel	Redis Cluster
数据分片	否	是，16384 槽
故障转移	主从切换	每个分片主从切换
客户端路由	查询当前主地址	维护槽到节点映射
多 key	单实例内正常	通常要求同槽
扩容	提升副本不增加容量	加节点并迁槽

21. Redis Cluster 的槽、重定向、Hash Tag 和扩缩容如何工作？

21.1 结论先行

Redis Cluster 把 key 空间划分为 16384 个 hash slot，每个主节点负责一部分槽。客户端根据 $CRC16(key) \bmod 16384$ 路由到节点；节点不会作为普通代理转发请求，而是返回 **MOVED** 或迁移期间的 **ASK** 重定向。

Cluster 同时解决分片与分片级高可用，但带来多 key 同槽、批量命令拆分、热点槽和扩缩容迁移等复杂度。

21.2 为什么使用槽而不是直接 key 到节点

槽是稳定的中间层：扩容时迁移槽及其 key，而不必为所有 key 维护全局映射。16384 个槽在管理开销和迁移粒度间折中。

```
slot 0..5000    -> master A
slot 5001..10000 -> master B
slot 10001..16383 -> master C
```

节点间通过 Cluster Bus 和 gossip 交换成员、故障和槽状态。

21.3 MOVED 与 ASK

MOVED

表示该槽已稳定归另一个节点：

```
-MOVED 3999 10.0.0.8:6379
```

客户端应更新槽缓存，后续直接请求正确节点。

ASK

迁槽期间，某些 key 已迁移、某些仍在源节点。**ASK** 是临时重定向，客户端向目标节点先发送 **ASKING** 再执行命令，但不永久更新槽归属。

成熟 Cluster 客户端会自动处理；手写客户端容易在迁槽时出错。

21.4 Hash Tag

Cluster 默认按完整 key 计算槽。花括号中的内容可作为 hash tag：

```
order:{1001}:detail
order:{1001}:items
```

二者按 **1001** 计算，落在同槽，可执行多 key Lua/事务。

Hash Tag 不能滥用。若所有订单都写 **{order}**，会全部落在一个槽，失去分片价值。tag 应选择能保持业务原子组且分布均匀的实体 ID。

21.5 跨槽限制

MGET、MSET、事务、Lua 和集合运算在开源 Cluster 中通常要求涉及 key 同槽。客户端可将独立操作按节点拆分并行执行，但这不是一个原子操作。

数据建模时应先定义原子边界：订单详情和订单状态需同槽；跨用户全局统计则应采用异步聚合或专门结构。

21.6 扩容与迁槽

新增主节点后，需要把一部分槽从旧节点迁移到新节点：

1. 将目标槽标记 importing/migrating。
2. 分批迁移槽内 key。
3. 迁移期间通过 ASK 路由。
4. 更新集群槽归属。

迁移风险取决于 key 大小和热点。一个槽包含大 Key 时迁移时间和网络占用会很高，因此大 Key 治理也是扩容能力的一部分。

21.7 缩容

先把待下线节点的所有槽迁移到其他主节点，确认无槽后删除节点。还要重新安排副本，避免主副本落在同一故障域。

21.8 槽不均衡与容量不均衡

平均槽数相等不代表内存和 QPS 相等。不同槽的 key 数、value 大小和热度差异很大。扩容应同时观察：

- 每节点 used_memory 和 RSS。
- key 数与过期 key 数。
- 读写 QPS、网络 and CPU。
- 槽级或 key 前缀的热点。

可按内存和热度迁槽，而不是只按槽数量平均。

22. Redis Cluster 能保证强一致吗，网络分区会发生什么？

22.1 结论先行

Redis Cluster 采用异步复制，目标是高性能、水平扩展和“合理但非强一致”的写安全。主节点确认的写在复制到副本前发生故障，可能丢失。网络分区时多数主节点所在侧可继续提供服务并完成故障转移，少数侧经过 `node-timeout` 后停止接受写。

Redis Cluster 不是 Raft/Paxos 共识数据库，不保证线性一致。

22.2 已确认写丢失的场景

```
client -> master A: SET k v, receives OK
A fails before replica receives write
replica A1 promoted
k=v is absent
```

这种窗口通常很短，但真实存在。客户端连到网络分区少数侧时，写丢失窗口可能更大，直到该侧发现无法联系多数主节点并停止写。

22.3 Cluster 可用条件

典型情况下，集群需要多数主节点可达，并且每个故障主节点有可提升的可达副本。若某槽没有可用主节点，默认配置下整个集群可能进入不可用状态；可通过配置调整部分槽故障时的服务策略，但应用需理解缺失槽的错误语义。

22.4 读副本的陈旧数据

Cluster 客户端可以显式从副本读取以扩展读吞吐，但读可能陈旧。故障切换时拓扑和复制状态变化，读己之写更难保证。可按业务使用主节点读、版本号或会话黏性。

22.5 如何缩小丢失窗口

- 配置足够副本，主副本分散故障域。
- 使用 `min-replicas-to-write` / `min-replicas-max-lag`。
- 关键写后使用 `WAIT`，接受额外延迟。
- 核心事实写数据库或一致性日志，Redis 保存派生状态。
- 故障演练测量真实 RPO，而不是只看架构图。

这些措施提高写安全但降低网络异常时的可用性，体现 CAP 取舍。

22.6 Cluster 的最后写入胜出

故障切换后，被选中的新主数据集成为权威，旧主恢复后会同步新主。Redis 不尝试合并两个分区上的冲突写。对需要多主写和冲突合并的跨地域场景，应评估带 CRDT 的 Active-Active 产品或其他数据库，而不是让两个开源 Cluster 主节点同时写同一份逻辑数据。

22.7 高质量回答

Redis Cluster 通过槽分片和副本提升获得扩展性与可用性，但复制是异步的，因此存在已确认写丢失窗口。多数侧能继续服务，少数侧最终拒写。WAIT 和最小副本配置只能缩小风险，不能把它变成共识系统。严格核心数据应由数据库约束或一致性日志兜底。

第四部分：内存、性能与 Java 工程实践

23. Redis 内存为什么会碎片化，如何做内存优化与容量规划？

23.1 结论先行

Redis 内存问题不能只看 `used_memory`。进程 RSS 还受到分配器碎片、客户端缓冲、复制 backlog、持久化 COW、Lua/模块和操作系统页缓存影响。容量规划应以峰值 RSS、故障切换和持久化期间的最坏情况为基准，而不是以当前业务 value 总大小为基准。

23.2 关键指标

INFO memory 中常关注：

- `used_memory`：Redis 分配器统计的已用内存。
- `used_memory_rss`：操作系统看到的驻留集。
- `used_memory_dataset`：数据集估算。
- `mem_fragmentation_ratio`：RSS 与已分配内存关系的粗略指标。
- `allocator_*`：分配器 active、resident、allocated。
- `mem_clients_normal`、`mem_clients_slaves`：客户端和副本缓冲。
- `mem_not_counted_for_evict`：部分不计入淘汰的内存。

碎片率大于 1 不一定是坏事，小实例固定开销会放大比例；小于 1 可能表示部分内存被换出，通常更危险。要结合绝对字节数、Swap 和延迟分析。

23.3 碎片化的来源

Redis 大量对象动态创建、扩容、删除。jemalloc 按 size class 和 arena 管理内存，释放的小块未必能立即组合并归还操作系统：

```
page: [used][free][used][free]
```

逻辑数据减少后 `used_memory` 下降，但 RSS 可能保持高位。不同大小对象频繁交替、TTL 批量过期和大规模 key churn 会加剧碎片。

23.4 Active Defragmentation

Redis 可启用主动碎片整理，逐步迁移对象以释放更完整的页。它用 CPU 换 RSS 改善，应通过阈值和 CPU 上限控制。高峰期激进整理可能增加延迟，因此需要观察：

- 碎片绝对浪费是否足够大。
- CPU 是否有余量。

- 延迟 SLO 是否允许。
- 整理后 RSS 是否真正下降。

有时重启副本并滚动切换比在线整理更可预测，但必须确保高可用和数据安全。

23.5 数据建模优化

- 缩短过长 key 名，使用清晰但紧凑的前缀。
- 小对象可适度合并到 Hash，利用紧凑编码；不要形成超级 Hash。
- 选择合适序列化：JSON 可读但冗余，Protobuf/MessagePack 更紧凑但调试复杂。
- 控制 value 大小和集合元素数量，建立硬上限。
- 删除使用 UNLINK 降低主线程释放阻塞。
- 对可近似统计使用 HLL/Bloom 等结构。
- 不在同实例混放生命周期和淘汰策略完全不同的数据。

23.6 容量估算

基础估算：

```
logical_data = key_count * avg(key + value + object/entry overhead)
steady_memory = logical_data / target_utilization
peak_rss = steady_memory
            + replication_and_client_buffers
            + aof_rewrite_buffers
            + fragmentation_reserve
            + cow_peak
            + process_reserve
```

对象开销随类型、编码、key 长度和分配器变化，不能只用理论公式。最可靠方法是生成与生产分布相似的数据集，在目标 Redis 版本和配置上测量 MEMORY USAGE、总 used_memory 和持久化峰值。

23.7 推荐水位

不应让稳定 used_memory 长期贴近物理内存。对开启持久化、写入活跃的实例，常把数据水位控制在物理内存的 50%-70% 区间，再通过实测修正；纯缓存、无 fork 的实例可更高，但仍需留客户端缓冲和碎片空间。

这不是统一标准数字。面试中应说明：水位由写入率、COW、对象分布、恢复策略和宿主机是否独占决定。

23.8 内存泄漏还是合理增长

排查步骤：

1. 看 key 数、每 DB key/expires 数是否增长。
2. 按前缀采样，找业务数据无 TTL 或异常维度。

3. 看客户端输出缓冲、blocked clients 和 pub/sub 慢消费者。
4. 看 backlog、AOF rewrite 和 Lua 内存。
5. 比较 `used_memory` 与 RSS，判断碎片或 COW。
6. 使用 MEMORY DOCTOR 获取提示，但不能替代指标分析。

24. Redis 延迟突然升高或 CPU/内存异常时如何系统排查？

24.1 排查总原则

先分清是客户端延迟、Redis 命令执行、网络、宿主机还是依赖系统问题。使用同一时间轴关联 P50/P99、QPS、命令、CPU、RSS、fork、磁盘和网络，避免看到一个指标就修改配置。

确认影响范围

- > 客户端还是服务端
- > 单节点还是全集群
- > 单命令/单 key 还是所有请求
- > 前台命令还是后台持久化
- > 止损
- > 证据闭环
- > 根因修复与演练

24.2 第一步：确认客户端侧

- 连接池等待时间是否升高。
- DNS、TCP 建连、TLS 和跨可用区 RTT。
- 客户端线程池/Netty event loop 是否被阻塞。
- JVM GC、CPU 和反序列化耗时。
- 超时配置是否把排队时间算在内。
- 是否出现重试风暴。

若服务端 SLOWLOG 很干净但客户端耗时高，应重点检查网络、连接池和响应体大小。

24.3 第二步：Redis 核心状态

```
INFO server
INFO clients
INFO memory
INFO stats
INFO commandstats
INFO latencystats
INFO persistence
INFO replication
INFO cpu
```

重点观察：

- `instantaneous_ops_per_sec` 与网络字节率。
- `blocked_clients`、`connected_clients`、拒绝连接。
- `used_memory`、RSS、Swap、碎片。
- `evicted_keys`、`expired_keys`、命中率。
- AOF/RDB 是否进行、最近 fork 时间、持久化错误。

- 主副本 link、offset 和全量同步。
- 每类命令调用次数、总耗时和错误数。

24.4 SLOWLOG 的正确理解

```
SLOWLOG GET 128
SLOWLOG LEN
```

SLOWLOG 记录命令真正执行所花时间，不含网络 I/O 和等待服务端读取请求的时间。阈值单位通常是微秒配置。它适合发现慢脚本、大集合命令和复杂查询，但无法证明网络无问题。

还需警惕“命令每次 1ms，不进慢日志，但单实例每秒几万次使 CPU 饱和”。应结合 commandstats 的累计耗时。

24.5 Latency Monitor

启用适当阈值后使用：

```
LATENCY LATEST
LATENCY HISTORY event
LATENCY DOCTOR
```

可帮助识别 fork、AOF fsync、过期、淘汰、命令和调度相关峰值。诊断功能本身也要按生产规范配置。

24.6 常见根因与证据

慢命令或大 Key

证据：SLOWLOG、commandstats、特定节点 CPU 高、网络大响应。处理：限流/禁用命令、分页、拆 key、异步删除、改模型。

fork / COW

证据：延迟峰值与 latest_fork_usec、BGSAVE/BGREWRITEAOF 同时发生，RSS 上升。处理：降低实例内存、错峰、预留内存、优化磁盘。

AOF fsync 抖动

证据：AOF pending/failure、磁盘 await 和队列升高。处理：检查磁盘、容器 IO 限额、云盘突发额度和 fsync 策略。

过期或淘汰风暴

证据：expired_keys/evicted_keys 激增、主动过期 CPU、命中率下降。处理：TTL 抖动、扩容、优化淘汰策略和写入节奏。

Swap / 宿主机争用

证据：Redis RSS 大于可用内存、major fault、swap in/out、steal CPU。处理：隔离宿主机、限制超卖、降低水位。

客户端输出缓冲

证据：CLIENT LIST 中 omem 大、Pub/Sub 或副本慢。处理：修复慢消费者、限制 buffer、降低大响应。

Cluster 热点

证据：单节点 CPU/网络高而其他节点空闲。处理：找热 key/槽、拆分、本地缓存、迁槽或扩容。

24.7 CPU 100% 排查

```
redis-cli --stat
redis-cli --latency
redis-cli --latency-history
redis-cli --bigkeys
redis-cli --memkeys
```

--bigkeys/--memkeys 会扫描键空间，应在低峰、从副本或隔离环境使用，并控制速率。还可使用系统 top, pidstat, perf 等确认 CPU 在用户态命令、内核网络还是后台线程。

24.8 应急止损

- 网关对异常接口限流。
- 临时禁用高危功能或降级为旧缓存/静态数据。
- 对大响应设置条数上限。
- 将流量从异常节点迁移，但避免同时触发全量复制风暴。
- 扩容前确认瓶颈可被分片，不能用加节点掩盖单热 key。
- 保护数据库，避免缓存降级引发二次事故。

24.9 复盘与防复发

根因修复应包括：命令复杂度审查、key 大小预算、TTL 规范、客户端超时和重试规范、容量阈值、持久化演练、故障注入和仪表盘。只“重启 Redis 后恢复”不算闭环。

25. Java 客户端连接、序列化、超时、重试和故障切换如何设计？

25.1 客户端选择

官方文档中，Jedis 主要提供直观的同步接口；Lettuce 支持同步、异步和 Reactive，并基于 Netty。Redisson 在客户端之上提供分布式锁、集合和同步器等高级抽象。

选择要看调用模型和团队能力：

- 简单同步服务：Jedis 易理解，必须正确使用池化或新客户端连接管理。
- Spring Data Redis/异步与 Reactive：Lettuce 常见。
- 需要成熟分布式对象封装：Redisson，但仍要理解底层故障语义。

不要仅因“Lettuce 连接线程安全”就让一个连接承载无限并发。连接的命令排队、阻塞操作和大响应仍会互相影响。

25.2 连接与池化

Jedis

传统 Jedis 连接通常不可被多个线程同时随意共享，需要使用连接池或新版客户端提供的连接管理。池参数应按并发、单请求 Redis 次数和延迟估算：

```
required_connections ~= peak_concurrent_redis_calls  
                      = request_rate * redis_call_latency * calls_per_request
```

池过小导致等待，过大导致 Redis 连接、内存和上下文开销上升。必须监控 active、idle、waiters 和 borrow latency。

Lettuce

普通非阻塞命令可复用连接，但阻塞命令、事务、Pub/Sub 和长 Pipeline 应使用独立连接。异步 API 只有在调用链不立刻 get() 阻塞时才有价值。

25.3 超时分层

至少区分：

- DNS/连接超时。
- TLS 握手超时。
- 命令超时。
- 连接池获取超时。

- 上层接口总 deadline。

上层 deadline 应覆盖所有重试和下游，不要让 Redis 客户端单次超时比接口总超时还长。超时太短会把正常抖动变成重试风暴；太长会耗尽线程池和连接池。

25.4 重试原则

可安全重试

- 纯读取，且允许重新路由。
- SET key deterministicValue 等幂等写。
- 带请求 ID 和状态查询的业务操作。

不可盲目重试

- INCR, LPUSH, ZINCRBY 等非幂等增量操作。
- Lua 已可能执行成功但客户端超时。
- 事务 EXEC 后响应丢失。

网络超时表示“结果未知”，不是“服务端没执行”。应用应使用幂等 key、业务流水和查询接口确认。

25.5 序列化设计

Java 原生序列化通常不推荐：体积大、跨语言差，并存在安全和类版本问题。常见选择：

- JSON：可读、兼容好，体积较大。
- Protobuf：紧凑、跨语言、需要 schema。
- MessagePack/Kryo：性能和体积好，但版本治理要严格。
- 原始 String/数字：简单场景最可靠。

value 应包含 schema version，支持灰度期间新旧版本共存。缓存反序列化失败要能删除并回源，而不是持续报错。

25.6 Cluster 拓扑刷新

成熟客户端维护槽映射并处理 MOVED/ASK。生产需启用合适的周期/自适应拓扑刷新，配置多个种子节点，并处理节点地址在容器、NAT 或 TLS 环境中的可达性。

故障切换期间：

- 旧连接可能收到连接关闭、READONLY 或重定向。
- 客户端刷新拓扑并建立新连接需要时间。
- 大量实例同时重连可能造成连接风暴。

使用带抖动的退避，设置最大重试次数，并让上层熔断。

25.7 Pipeline 和异步背压

异步并不意味着无限并发。若每秒提交百万个 Future 而不限制未完成请求，客户端堆内存和 Redis 输出缓冲都会增长。应设置 in-flight 上限、批次大小和响应消费速率。

25.8 Spring Cache 常见问题

- 默认 key 生成不稳定或冲突。
- 没有区分“未命中”和缓存空值。
- 缺少 TTL 或所有缓存统一 TTL。
- @Cacheable 同类内部调用绕过代理。
- 方法返回大对象或包含不可序列化字段。
- 只使用缓存注解却没有数据库一致性方案。

缓存注解是调用封装，不会自动解决击穿、可靠失效和故障降级。

25.9 Redisson 锁注意事项

Redisson 看门狗在未指定固定 lease time 时可自动续期，官方文档默认 watchdog timeout 为 30 秒且可配置。但 JVM 暂停、调度延迟或网络问题仍可能续期失败。业务仍需 try/finally 解锁、检查当前线程持锁，并对严格资源使用 fencing/唯一约束。

25.10 Java 示例：带幂等结果的缓存加载

```
public ProductView getProduct(long id) {
    String key = "product:" + id;
    byte[] cached = redis.get(key);
    if (cached != null) {
        return decode(cached);
    }

    // Local single-flight or a short distributed rebuild lease.
    return singleFlight.execute(key, () -> {
        byte[] secondCheck = redis.get(key);
        if (secondCheck != null) {
            return decode(secondCheck);
        }

        ProductView value = repository.findView(id);
        if (value == null) {
            redis.setEx(key, jitterSeconds(30), NULL_MARKER);
            return null;
        }

        redis.setEx(key, jitterSeconds(900), encode(value));
        return value;
    });
}
```

实际代码还应区分 NULL_MARKER、设置总 deadline、限制重建并发并在 Redis 故障时回源限流。

第五部分：综合系统设计

26. 场景设计：构建高并发商品缓存、库存与异步事件系统

26.1 题目

设计一个电商活动系统：

- 商品详情峰值读 QPS 50 万。
- 10 万件库存，瞬时抢购请求 200 万。
- 每个用户限购 1 件，不能超卖。
- 下单后需要异步创建订单、扣数据库库存并发送通知。
- Redis 故障、主从切换、消息重复和缓存雪崩时系统要可控。

要求说明缓存、限流、库存预扣、消息、数据库一致性、高可用和故障补偿。

26.2 总体架构

```

CDN / static page
-> API Gateway: token, captcha, rate limit
-> Seckill Service
    -> local cache
    -> Redis Cluster
        product cache
        user eligibility
        Lua stock reservation
        event Stream / outbox bridge
-> Order Consumer
    -> relational DB transaction
    -> notification / downstream MQ
-> reconciliation and compensation jobs
    
```

核心原则：入口先削峰；Redis 只做高并发资格和预扣；数据库唯一约束与条件更新是最终事实；消息至少一次，消费者幂等；所有状态可对账和补偿。

26.3 商品详情缓存

Key 设计

```

product:{productId}:view
product:{productId}:version
    
```

详情按商品 ID 分布。热点商品通过 CDN、本地缓存和 Redis 多级承载。缓存 value 带版本和逻辑过期时间。

读路径

1. CDN/边缘返回静态部分。
2. 应用本地缓存返回极热点。
3. Redis 查询动态详情。

- 4. miss 时 SingleFlight, 只允许少量请求回源。
- 5. 返回旧值并异步刷新, 或在无旧值时受控等待。

更新

数据库提交后写 outbox, CDC/事件消费者删除 Redis 并发布本地缓存失效。TTL 带抖动作为最终收敛。

26.4 入口防刷与限流

- 活动前生成短期动态路径或签名 token。
- 用户、设备、IP 和接口多维限流。
- 网关本地令牌桶先挡住明显洪峰。
- Redis 维护跨实例用户资格和全局配额。
- 超过系统承载量快速返回“排队/售罄”, 不能让所有请求进入库存脚本。

验证码和动态路径不是安全边界, 只是提高攻击成本。

26.5 Redis 数据模型

使用同一商品 hash tag 保证原子 key 同槽:

seckill:{sku1001}:stock	String integer
seckill:{sku1001}:buyers	Set or request state Hash
seckill:{sku1001}:events	Stream
seckill:{sku1001}:reservation:*	optional result/status

所有商品若使用同一个 tag 会形成热点, 因此 tag 必须是 SKU ID。单个超级热门 SKU 仍是单槽热点, 需要入口限流和库存令牌分段。

26.6 原子资格检查与预扣

Lua 逻辑:

```
-- KEYS[1] stock, KEYS[2] buyers, KEYS[3] stream
-- ARGV[1] userId, ARGV[2] requestId, ARGV[3] timestamp

if redis.call('SISMEMBER', KEYS[2], ARGV[1]) == 1 then
    return {2, 'DUPLICATE'}
end

local stock = tonumber(redis.call('GET', KEYS[1]) or '-1')
if stock <= 0 then
    return {0, 'SOLD_OUT'}
end

redis.call('DECR', KEYS[1])
redis.call('SADD', KEYS[2], ARGV[1])
local messageId = redis.call('XADD', KEYS[3], '*',
    'requestId', ARGV[2],
    'userId', ARGV[1],
    'ts', ARGV[3])
return {1, messageId}
```

限制脚本只做短操作。返回结果要可按 requestId 查询，处理客户端超时后的未知状态。

26.7 热点库存的进一步拆分

单 SKU 200 万瞬时请求仍可能压垮一个主线程。可在入口将 10 万库存拆为多个令牌桶:

```
stock:{sku1001}:00 ... stock:{sku1001}:63
```

请求按用户 hash 路由分片。每个分片有配额，售罄不均时后台在分片间再平衡。这样牺牲了一些即时利用率，换取并行吞吐。最终库存总和必须受数据库约束和对账。

另一方案是活动层先发放有限排队令牌，只有拿到令牌请求进入单槽原子扣减，从源头限制脚本 QPS。

26.8 订单消费

消费者读取 Stream:

```
XREADGROUP -> DB transaction -> XACK
```

数据库事务:

```
INSERT INTO orders(request_id, user_id, sku_id, status)
VALUES (?, ?, ?, 'CREATED');

UPDATE sku_stock
SET available = available - 1
WHERE sku_id = ? AND available > 0;
```

约束:

- request_id 唯一，防消息重复。

- (activity_id, user_id, sku_id) 唯一，防重复购买。
- 库存条件更新影响行数必须为 1，最终防超卖。
- 订单和数据库库存同事务提交。

若数据库库存失败，记录补偿事件，把 Redis 预扣库存归还并清除/更新购买状态。补偿也必须幂等，不能重复加库存。

26.9 Redis Stream 与外部 MQ

若 Stream 只用于活动内部短链路，可使用消费组和 PEL。若订单事件还要长期回放、跨团队和跨地域，建议订单成功后通过数据库 Outbox 发布到 Kafka/RabbitMQ。Redis 预扣事件必须有归档或重建机制，不能让内存裁剪删除唯一证据。

26.10 故障场景

Redis 主节点故障

Cluster 副本提升期间短暂错误。客户端刷新拓扑，入口降速。异步复制可能丢少量已确认预扣，因此数据库唯一约束和用户结果查询必须兜底；活动可选择故障时 fail-closed，暂停新预扣。

Lua 成功但客户端超时

客户端携带 requestId 查询预扣结果，不能直接重复扣减。脚本先检查 requestId/用户状态并返回相同结果。

Stream 消费者宕机

消息留在 PEL，超过可见性时间由其他消费者 claim。数据库唯一约束保证重复处理安全。

数据库不可用

停止或降低消费，Stream 暂时积压。设置最大积压和活动入口熔断，避免 Redis 内存耗尽。恢复时按数据库能力限速消费。

缓存整体失效

本地旧值、静态页和限流继续工作；数据库回源有独立并发上限。先恢复关键热点，而不是全量并发预热。

26.11 对账与补偿

定期核对：

```
initial_stock
= redis_remaining
+ successful_orders
+ valid_reservations_in_processing
+ compensated_failures
```

对账表按 requestId 保存 Redis 预扣、Stream 消息、数据库订单和补偿状态。发现差异后执行幂等修复，并保留审计记录。

26.12 容量规划

- 详情缓存：key 数、平均 value、TTL、热点带宽。
- 买家 Set：10 万成员的实际内存和迁移成本。
- Stream：峰值消息大小、消费延迟和最大积压。
- 客户端连接：应用实例数 x 每实例连接。
- Cluster：单节点脚本 QPS、网络 and CPU，而不是只看总容量。
- 持久化：活动写入时 COW 和 AOF rewrite 峰值。

活动前必须进行近真实数据分布压测、故障切换演练和数据库降级演练。

26.13 面试总结模板

我会把商品读、资格校验、库存事实和订单事件分层。CDN 和本地缓存承担读热点，网关先削峰；Redis Lua 原子完成一人一单和预扣，requestId 处理未知结果；数据库唯一约束和条件更新最终防重与防超卖；消费至少一次但业务幂等；Redis、消息和数据库之间通过状态机、Outbox、对账和补偿实现最终一致。对于单 SKU 热点，我会先限制进入 Redis 的请求量，再考虑令牌分片，而不是仅靠加 Cluster 节点。

附录 A：高频追问速答

A.1 Redis 为什么不用多线程执行所有命令？

核心数据操作短、内存访问快，串行模型降低锁和切换成本；CPU 饱和时通过分片扩展。现代 Redis 仍有 I/O 线程、后台线程和子进程，所以“Redis 完全单线程”不准确。

A.2 SETNX 能否直接做锁？

不能只用 SETNX。必须原子设置 TTL，value 唯一，解锁比较 value 后删除；严格场景还需 fencing token。

A.3 DEL 和 UNLINK 的区别？

DEL 在主线程释放对象，删除大对象可能阻塞；UNLINK 从 keyspace 摘除后后台释放，通常更适合大 Key。两者都不是跨系统事务。

A.4 EXPIRE 到点会立即删除吗？

语义上到期后不可见，物理删除由被动访问和主动采样完成，内存不一定立即下降。

A.5 SCAN 是否会漏数据？

对遍历期间不变化的数据能完成遍历；数据变化时可能重复，也不能把它当一致性快照。客户端需去重并容忍变化。

A.6 Redis 能保证消息不丢吗？

Pub/Sub 不能；Streams 配合持久化和消费组可提高可靠性，但 Redis 故障、异步复制和跨系统提交仍有窗口。核心消息需要数据库 Outbox、专业 MQ 或对账。

A.7 缓存命中率越高越好吗？

不一定。低价值大对象可能提高命中率却增加延迟和成本。应按业务看命中带来的数据库减载、P99、数据新鲜度和内存收益。

A.8 主从切换是否无损？

通常不是。复制异步，存在已确认写尚未到副本的窗口。WAIT 和最小副本配置只能缩小风险。

A.9 Redis Cluster 为什么是 16384 个槽？

槽数是路由元数据、gossip 负担和迁移粒度之间的工程折中。客户端按槽而不是每个 key 维护映射。

A.10 为什么不建议所有 key 永久不过期？

数据只增不减，缓存污染和容量不可控；业务删除遗漏会成为长期垃圾。永久 key 应有明确生命周期和审计。

A.11 缓存和数据库双写能否用分布式事务？

通常不值得。缓存是可再生副本，采用数据库提交后失效缓存、可靠事件、TTL 和版本控制更合适。强一致数据不应依赖缓存。

A.12 如何判断 Redis 该扩容还是优化？

若是总内存、总 CPU、网络随业务均匀增长，可扩容；若是单热 key、大 key、慢脚本或高危命令，加节点可能无效，应先治理模型。

附录 B：常用排障命令速查

B.1 服务与统计

```
PING
INFO server
INFO clients
INFO memory
INFO stats
INFO commandstats
INFO latencystats
INFO persistence
INFO replication
INFO keyspace
```

B.2 延迟与慢命令

```
SLOWLOG GET 128
SLOWLOG LEN
LATENCY LATEST
LATENCY HISTORY command
LATENCY DOCTOR
redis-cli --latency
redis-cli --latency-history
redis-cli --stat
```

B.3 Key 与内存

```
MEMORY USAGE key SAMPLES 5
MEMORY STATS
MEMORY DOCTOR
OBJECT ENCODING key
TTL key
PTTL key
SCAN 0 MATCH prefix:* COUNT 1000
```

KEYS、DEBUG、全量大集合读取和高频 MEMORY USAGE 可能影响生产，应通过 ACL 限制并在受控环境使用。

B.4 客户端

```
CLIENT LIST
CLIENT INFO
CLIENT KILL ...
CLIENT PAUSE ...
```

重点关注连接数、idle、flags、输入/输出缓冲和阻塞状态。对生产连接执行 KILL/PAUSE 前必须评估影响。

B.5 复制与 Cluster

```
ROLE
INFO replication
WAIT 1 1000
CLUSTER INFO
CLUSTER NODES
CLUSTER SLOTS
CLUSTER SHARDS
CLUSTER KEYSLOT key
CLUSTER COUNTKEYSINSLOT slot
```

B.6 Streams

```
XINFO STREAM stream
XINFO GROUPS stream
XINFO CONSUMERS stream group
XPENDING stream group
XAUTOCLAIM stream group consumer min-idle start COUNT 100
```

附录 C：生产上线检查清单

C.1 数据模型

- 每类 key 有命名规范、owner、value 格式和最大大小。
- 集合有最大成员数和分页规则。
- TTL 有默认值和随机抖动，永久 key 有审批理由。
- Cluster 原子 key 的 Hash Tag 正确且分布均匀。
- 缓存空值、版本和 schema 升级策略明确。

C.2 客户端

- 连接池/连接复用经过峰值并发估算。
- 连接、命令、池等待和接口总 deadline 分层配置。
- 重试区分幂等与非幂等，带退避和抖动。
- Pipeline 有批次和 in-flight 上限。
- Cluster/Sentinel 拓扑刷新和故障地址可达性已验证。
- 序列化有版本号，不使用不安全 Java 原生序列化。

C.3 高可用与持久化

- 主副本跨主机/可用区，Sentinel 至少分散部署。
- 明确 RPO/RTO，并验证 AOF/RDB 配置。
- 备份复制到独立故障域并定期恢复。
- 主从切换、全量同步和冷启动已演练。
- min-replicas-*、WAIT 等写安全配置符合可用性取舍。

C.4 性能与容量

- 稳态内存为 fork、COW、碎片和缓冲预留空间。
- 大 Key、热 Key、慢命令和命中率按业务监控。
- 观察 fork、AOF fsync、复制 offset、eviction 和 rejected connection。
- 宿主机避免 Swap、CPU 超卖和慢网络盘。
- 压测包含真实 value 大小、TTL 分布、持久化和故障切换。

C.5 安全

- 使用 ACL 最小权限，不共享全能账号。
- 禁止公网直连，启用网络隔离和 TLS（按部署能力）。
- 限制 CONFIG、DEBUG、FLUSH、KEYS、EVAL 等危险命令。
- 密钥轮换和审计日志流程明确。
- 备份加密，凭证不写入代码和普通日志。

C.6 业务降级

- Redis 不可用时明确 fail-open 或 fail-closed。
- 缓存回源有并发上限，数据库有隔离和熔断。
- 热点可返回旧值、静态页或友好错误。
- 消息积压有阈值、死信和恢复限速。
- 锁、限流和幂等组件失败时业务语义明确。

附录 D：官方参考资料

以下资料用于校验本文中的版本差异、命令语义和故障边界。官方文档会持续更新，生产配置应以实际版本文档为准。

1. Redis data types: <https://redis.io/docs/latest/develop/data-types/>
2. Compare data types: <https://redis.io/docs/latest/develop/data-types/compare-data-types/>
3. OBJECT ENCODING: <https://redis.io/docs/latest/commands/object-encoding/>
4. Redis pipelining: <https://redis.io/docs/latest/develop/using-commands/pipelining/>
5. Transactions: <https://redis.io/docs/latest/develop/using-commands/transactions/>
6. Scripting with Lua: <https://redis.io/docs/latest/develop/programmability/eval-intro/>
7. Key expiration: <https://redis.io/docs/latest/commands/expire/>
8. Key eviction: <https://redis.io/docs/latest/develop/reference/eviction/>
9. Redis persistence: https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/
10. Redis replication: https://redis.io/docs/latest/operate/oss_and_stack/management/replication/
11. Redis Cluster specification: https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/
12. Distributed locks: <https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/>
13. Redis Streams: <https://redis.io/docs/latest/develop/data-types/streams/>
14. Diagnosing latency: https://redis.io/docs/latest/operate/oss_and_stack/management/optimization/latency/
15. Latency monitoring: https://redis.io/docs/latest/operate/oss_and_stack/management/optimization/latency-monitor/
16. Memory optimization: https://redis.io/docs/latest/operate/oss_and_stack/management/optimization/memory-optimization/
17. INFO command: <https://redis.io/docs/latest/commands/info/>
18. MEMORY USAGE: <https://redis.io/docs/latest/commands/memory-usage/>
19. SLOWLOG: <https://redis.io/docs/latest/commands/slowlog/>
20. Redis Open Source 8.0 release notes: https://redis.io/docs/latest/operate/oss_and_stack/stack-with-enterprise/release-notes/redisce/redisos-8.0-release-notes/
21. What's new in Redis 8.8: <https://redis.io/docs/latest/develop/whats-new/8-8/>
22. Jedis guide: <https://redis.io/docs/latest/develop/clients/jedis/>
23. Lettuce guide: <https://redis.io/docs/latest/develop/clients/lettuce/>
24. Redisson locks and synchronizers: <https://redisson.pro/docs/data-and-services/locks-and-synchronizers/>

结束语

Redis 面试的难点不在命令数量，而在于准确描述保证边界：单条命令原子但跨系统不原子，持久化降低丢失但不替代备份，副本和 Cluster 提高可用但不等于强一致，锁有租约但旧持有者仍可能继续运行，缓存提高吞吐但引入新鲜度和回源风险。

高级工程师的回答应始终落到四个问题：**数据的最终权威是谁、故障窗口是什么、系统如何降级、如何通过指标和演练证明方案有效。**